



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

COMPUTER SCIENCE AND ENGINEERING
SOFTWARE ENGINEERING 2

Design Document Students&Companies

Authors:

Marcello Braghioli
Filippo Valentini

Academic Year:
2024-2025

Deliverable: DD

Title: Design Document

Authors: Marcello Braghiroli, Filippo Valentini

Version: 1.0

Date: 06-January-2025

Download page: <https://github.com/marcellobraghiroli/BraghiroliValentini>

Copyright: Copyright © 2024, Marcello Braghiroli, Filippo Valentini –
All rights reserved

Contents

Contents	ii
-----------------	-----------

1 Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions, Acronyms and Abbreviations	2
1.3.1 Definitions	2
1.3.2 Acronyms	2
1.3.3 Abbreviations	2
1.4 Revision History	3
1.5 Reference Documents	3
1.6 Document Structure	3
2 Architectural Design	4
2.1 Overview	4
2.2 Component View	5
2.3 Deployment View	10
2.4 Runtime View	10
2.5 Component Interfaces	30
2.5.1 Application Server - exposed API	30
2.5.2 External APIs	37
2.5.3 Application Server - internal interfaces	38
2.5.4 Data Layer interfaces	46
2.6 Selected Architectural Styles and Patterns	50
2.7 Other Design Decisions	51
3 User Interface Design	53
4 Requirements Traceability	61

5	Implementation, Integration and Test Plan	70
5.1	Plan	70
5.2	Features Identification	70
5.3	Component Integration and Testing	71
5.4	System Testing	76
6	Effort Spent	78
7	References	79
	List of Figures	80
	List of Tables	82

1 | Introduction

1.1. Purpose

Finding the right company in which to carry out internships is an issue that a lot of students must handle, both to complete their university carrier and to firstly approach the workplace environment. Unfortunately, it is not always easy for students to find a company whose projects and activities match their interests and skills, especially when the number of available opportunities is high. From another point of view, companies must deal every day with the selection of CVs, basing on students' attitudes and experiences, in order to find suitable figures for the activities they offer.

Students&Companies (S&C) is a platform that helps the matching between university students looking for internships and companies that offer these opportunities. The main idea is that students can publish their CV on the platform to find internships that might interest them, while companies advertise their internships so that the system can help them find suitable students. Companies can also set up and plan interviews with the matching students through the S&C platform, which helps them in the selection process by collecting and analyzing students' answers through structured questionnaires. S&C enables also the interested parties to monitor, keep track and provide information about the execution of the matchmaking processes and the status of the related internships.

1.2. Scope

This document presents a detailed description of all the decisions that guided the design stage of the S&C application. The high level architectural style chosen for the system is the 4-Tier architecture, which guarantees a clear separation of concerns, high modularity and scalability of single components. Interaction between external tiers is facilitated through RESTful APIs, enabling standardized and stateless interactions and improving performance and maintainability. The data layer of the architecture has been designed exploiting the functionalities of MongoDB, in order to manage structured data in a flexible and efficient way and to provide powerful text-search capabilities, critical for the

recommendation process involving student CVs and internships. All core functionalities of the application (such as authentication, recommendation and notification) have been encapsulated in different components, following a modular approach and ensuring logical separation of concerns. The system will also interact with specialized external platforms, in order to manage the process of student enrollment and company identity verification through efficient and reliable mechanisms. A more detailed presentation of these architectural choices will be provided in the following chapters, together with a description of the implementation and integration plan of designed components and of the system testing.

1.3. Definitions, Acronyms and Abbreviations

1.3.1. Definitions

- **Users:** they include both students and companies.
- **Matchmaking process:** phase in which students and companies show interest for each other in the context of an internship, possibly basing on system recommendations.
- **Selection process:** phase in which companies decide the students to admit for their internships, basing on interviews and questionnaires outcomes.
- **Credentials:** user e-mail address and password.

1.3.2. Acronyms

- **S&C:** Students&Companies.
- **API:** Application Programming Interface.
- **UI:** User Interface.
- **CV:** Curriculum Vitae.

1.3.3. Abbreviations

- **R*:** functional requirement number *.
- **UC*:** use case number *.
- **F*:** feature number *.

1.4. Revision History

- Version 1.0 (06/01/2025)

1.5. Reference Documents

This document is mainly based on the following materials:

- RASD and DD assignment specification for the Software Engineering 2 course, a.y. 2024-2025.
- Slides of the Software Engineering 2 course, available on WeBeep.

1.6. Document Structure

To provide a better understanding of the document, we report its chapter division below:

1. **Introduction:** it briefly presents the project, describing the purpose, the main architectural choices and design decisions. It also provides some useful details for the understanding of the document.
2. **Architectural Design:** it contains a comprehensive analysis of the architecture of the system, giving an high level description and presenting different design views, along with the interaction between components.
3. **User Interface Design:** it includes the mockups of the main user interfaces.
4. **Requirements Traceability:** it shows the mapping between the requirements previously defined in the RASD and the design elements introduced in the previous chapters.
5. **Implementation, Integration and Test Plan:** it describes the order in which the implementation should proceed, along with the planning of components integration and their testing.
6. **Effort Spent:** it shows the time spent by each team member for the fulfillment of the project.
7. **References:** here additional references to the main mentioned above are listed.

2 | Architectural Design

2.1. Overview

As mentioned in the introduction, S&C will adopt a **4-tier architecture**, represented at a high level in Figure 2.1. This choice fully enables the separation of the logic of the layers (presentation, business and data), also ensuring scalability, modularity and maintainability. Indeed, each tier component can be scaled independently from the others, improving availability. Also, faults remain isolated, and upgrades or changes can be made without affecting other elements. Moreover, the decision of having a web server was made to decouple browsers and application server, improving performance and security.

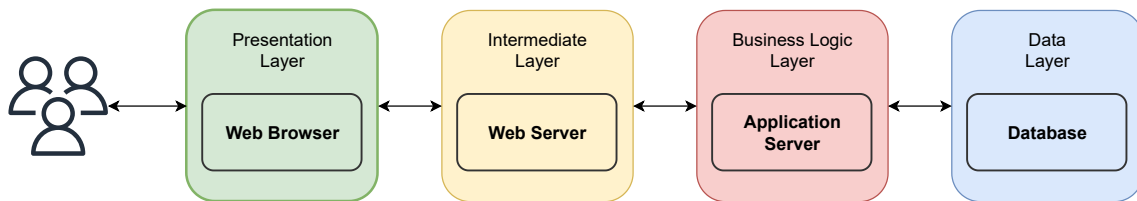


Figure 2.1: High-level architectural schema

Below, a brief description of the four tiers is provided.

1. The **first tier** of the architecture is represented by the web browser, the client application that allows users to interact with the system. It shows the content of the application and sends HTTPS requests towards the web server.
2. As **second tier**, the web server acts as an intermediary between the web browser and the application server. It serves static assets improving performances and routes dynamic requests to the appropriate application server endpoints. It also improves security isolating the backend and performing HTTPS requests filtering.
3. The **third tier** is the application server, that implements the business logic of the application, receiving request from the web server and interacting with the data layer.

4. The **fourth** and final **tier** is the database, that stores and handles data for the application. Thanks to the chosen architecture, it remains isolated and is only accessed by the application server.

Clients and servers (both web and application) interact with each other through flexible RESTful APIs, which are simple and standardized, in addition to being stateless and therefore improving scalability.

S&C will also implement replication of each of the mentioned components, supplemented by non-logical elements such as firewalls and load balancers. However, details about the system deployment are provided in section 2.3.

2.2. Component View

The component diagram of the system is shown in Figure 2.2. A brief description of the components is provided below.

First tier (Web Browser)

- **Web Browser:** it is the client application, that enables users to interact with the system.

Second tier (Web Server)

- **Web Server:** it acts as an intermediary between the web browser and the application server, handling HTTPS requests (also implementing security measures such as parsing and sanitizing), serving static assets and routing dynamic requests to the application server, through three interfaces offered by the Authentication Manager, the Student Manager and the Company Manager.

Third tier (Application Server)

- **Authentication Manager:** it handles user authentication, implementing login and registration functionalities. In addition, it interacts with the external platforms for student and company identity verification, at the time of registration.
- **Student Manager:** it handles incoming student requests, interacting directly with the data layer for simple operations (such as student information retrieval or CV upload), and with the other components to decompose more complex tasks.
- **Company Manager:** it handles incoming company requests, interacting directly

with the data layer for simple operations (such as company information retrieval or internships publication), and with the other components to decompose more complex tasks. The choice of having three components exposing external interfaces enables logical separation of concerns, also improving maintainability.

- **Recommendation Manager:** it handles the recommendation process, managing recommended CVs and internships and computing them when required, in addition to creating and evaluating satisfaction questionnaires regarding completed matches and active internships.
- **Matching Manager:** it handles the matchmaking process between students and companies, retrieving and updating ongoing and completed matches.
- **Selection Manager:** it handles the selection process after a match has been completed, with operations such as interview set up and retrieving, selection process questionnaires creation or compilation and student admission/rejection.
- **Notification Manager:** it handles notifications, which are delivered via email to students and companies. Notification requests are issued by the Student Manager and the Company Manager.
- **Messaging Manager:** it handles messages between students and companies in the context of active internships, receiving requests from the Student and Company managers.
- **Data Access Manager:** it is a centralized component handling interactions with the database. It abstracts queries to other components, improving separation of concerns, modularity and maintainability.

Fourth tier (Database)

- **DBMS:** it is the Database Management System, responsible for handling data access. The chosen technology is MondoDB, suitable for efficient and scalable storage of structured and unstructured data in JSON format. It also offers full-text search capabilities (Atlas Search), particularly useful in the recommendation process.

External Components

- **Student Verification Platform:** it is the external platform offering the API to the Authentication Manager for student authentication through their institutional credentials.
- **Company Verification Platform:** it is the external platform, whose offered API

is exploited by the Authentication Manager for the company identification through their business registration number.

Interfaces

For the sake of readability, component interfaces' names have not been included in the component diagram, but are shown below. The numbers correspond to the ones in Figure 2.2.

1. WebServerInterface.
2. AuthenticationManagerInterface.
3. StudentManagerInterface.
4. CompanyManagerInterface.
5. NotificationManagerInterface.
6. RecommendationManagerInterface.
7. MatchingManagerInterface.
8. SelectionManagerInterface.
9. MessagingManagerInterface.
10. DataAccessManagerInterface.
11. DBMSInterface.
12. StudentVerificationPlatformInterface.
13. CompanyVerificationPlatformInterface.

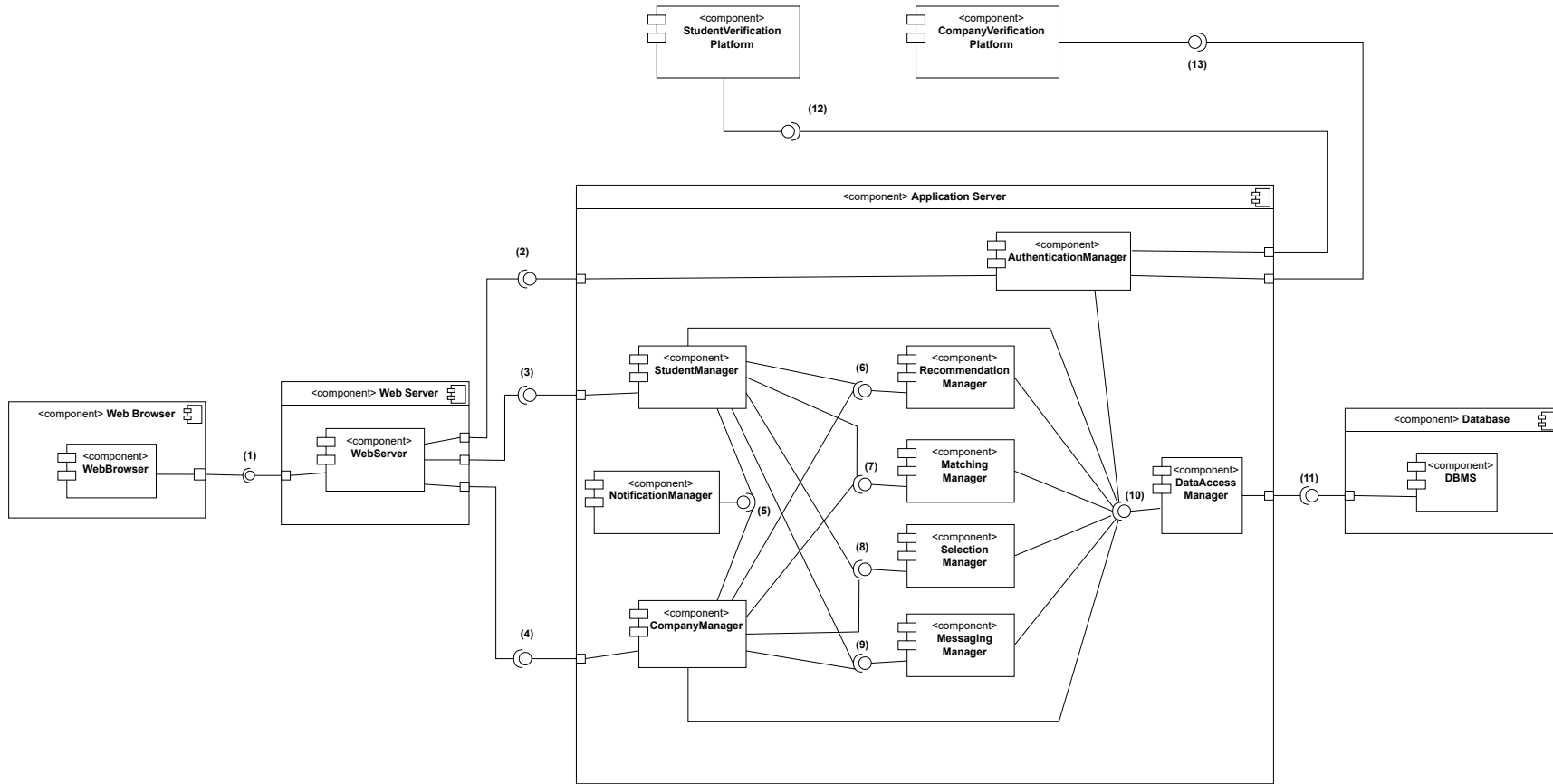


Figure 2.2: Component Diagram

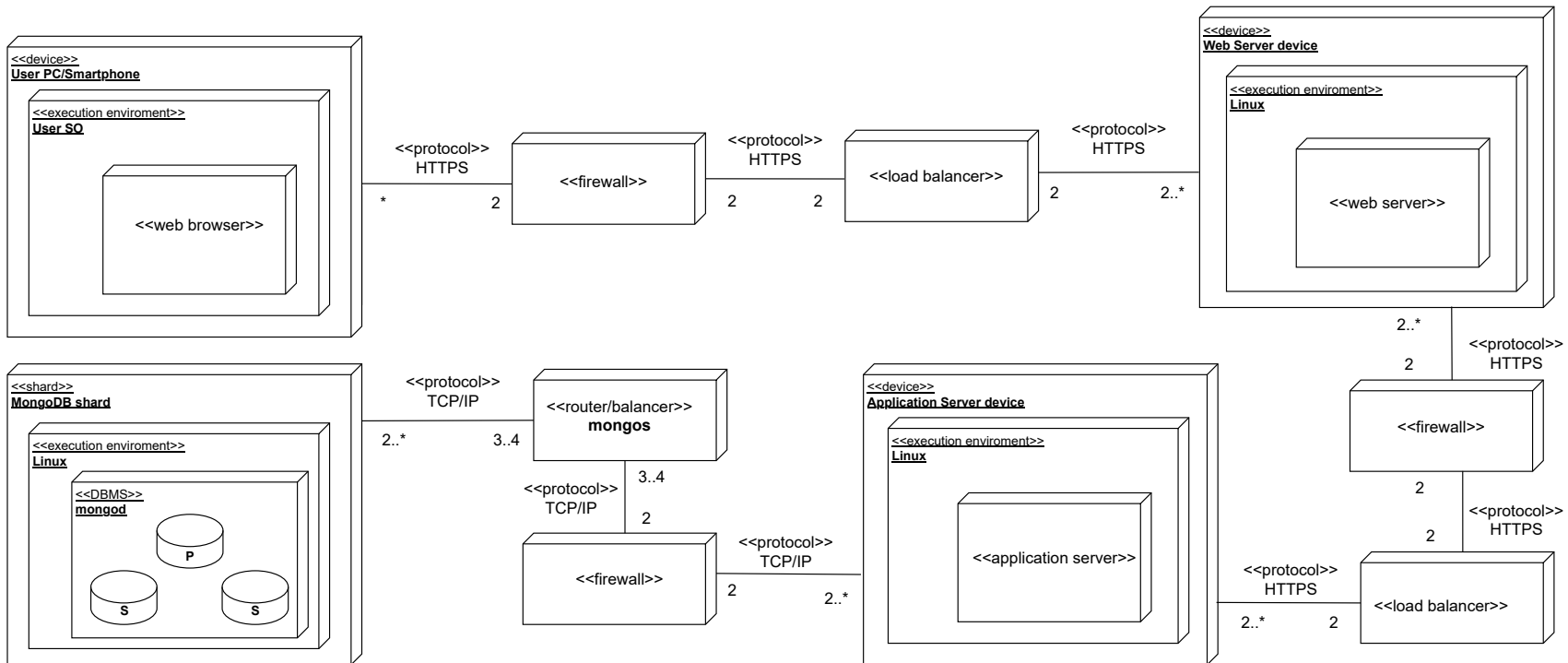


Figure 2.3: Deployment Diagram

2.3. Deployment View

S&C architecture deployment has been designed to meet non-functional requirements, especially regarding performance, availability and reliability. Figure 2.3 shows the deployment diagram, including non-logical elements. In particular, firewalls have been placed in strategic locations to control, filter and monitor traffic. In addition to that, load balancers play a crucial role, evenly distributing requests across server replicas to reduce overload on them and enhance performance, and routing traffic around failures to ensure high availability. Both firewalls and load balancers are deployed with two replicas to avoid single point of failure, keeping the application reliable. Moreover, to satisfy requirements regarding uptime and response time, as well as maintainability, both web servers and application servers will adopt auto-scaling mechanisms, with a minimum of two replicas for basic scalability, to dynamically adjust the number of instances based on current workload. Finally, a MongoDB cluster is used as data storage. Distribution and replication are automatically and independently handled by mongos instances, that act as request router and balancer as well. Note that, to enable sharding, a minimum of two shards is required. As concerns communication protocols, HTTPS is used from web browser until application servers, enabling TLS data encryption. To connect to the database, the system uses the MongoDB proprietary Wire Protocol, operating over classical TCP/IP.

2.4. Runtime View

This section is aimed at giving an understanding of S&C runtime behavior. For each of the use cases identified in the RASD, a sequence diagram illustrating interactions between components is shown. Note that, for simplicity and readability, the WebServer has not been included in the diagrams, since it acts as an intermediary between the WebBrowser and the Application Server. Also, interactions between the DataAccessManager and the DBMS should be MongoDB queries, but are shown as simple methods in the diagrams and in the following section regarding component interfaces, as inserting real queries would have resulted in a much less clear representation of the workflow.

[UC1] - Student registration

Figure 2.4 shows student registration. To complete it, students first need to authenticate through an external platform using their university institutional credentials. The AuthenticationManager acts as an intermediary between the WebBrowser and the StudentVerificationPlatform, forwarding requests and redirecting the student to the right page. After the authentication, students insert their new S&C credentials and the system

checks whether the inserted e-mail address is already used, before saving student information and finalizing the registration. Whether in the case of failed authentication or already used email, the system shows an error message to students.

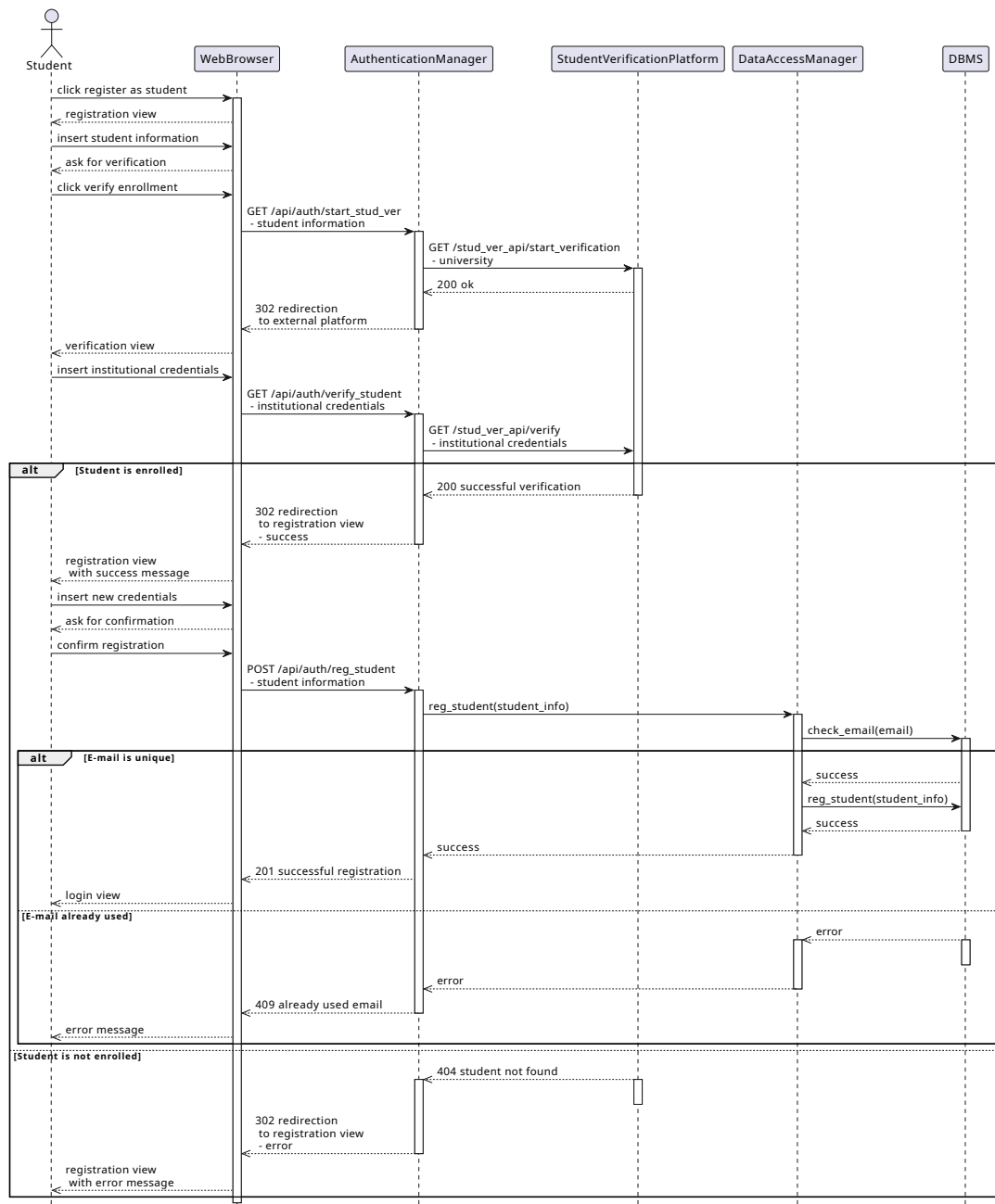


Figure 2.4: Student registration Sequence Diagram

[UC2] - Company registration

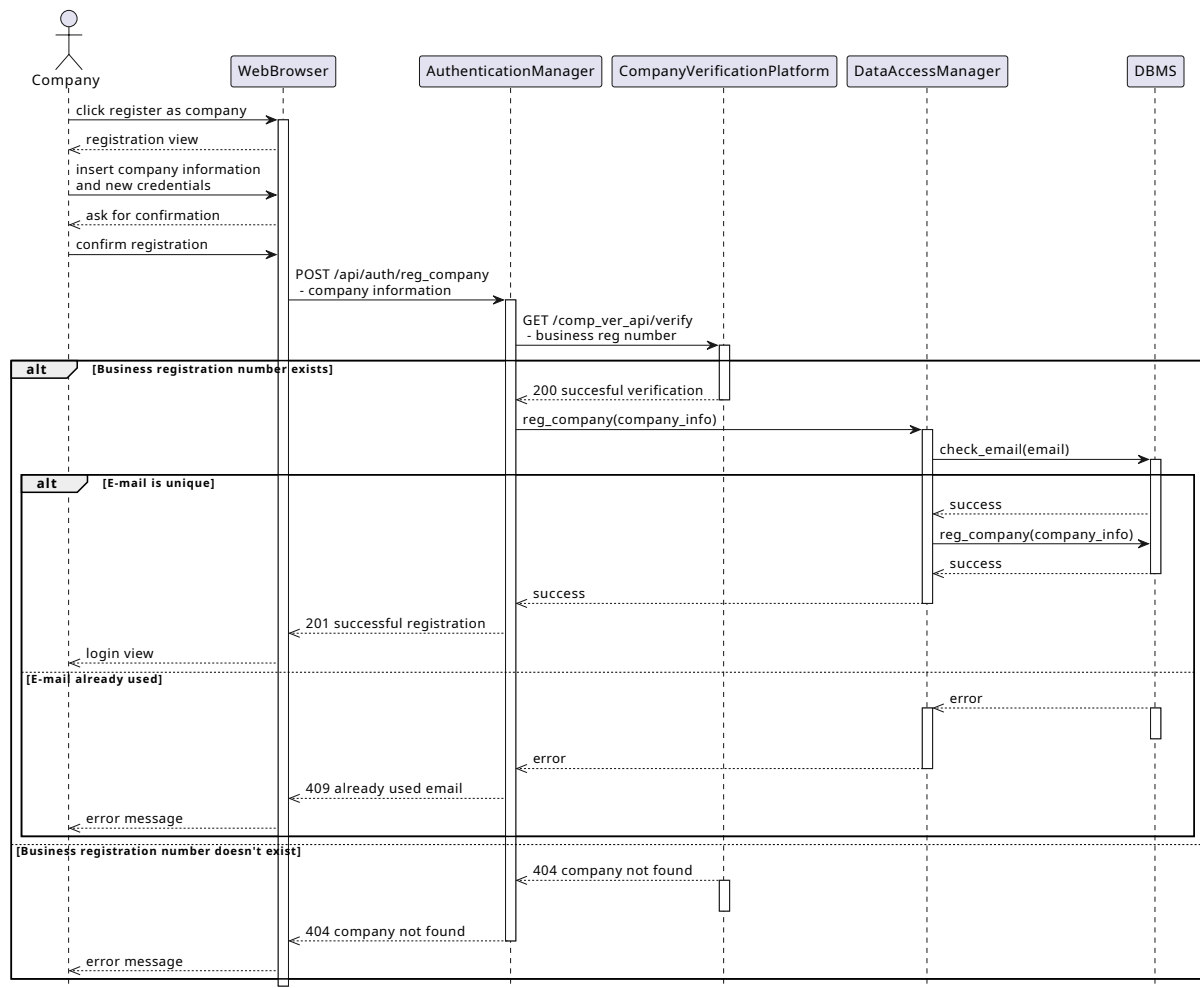


Figure 2.5: Company registration Sequence Diagram

Figure 2.5 shows company registration. Unlike students, companies insert all their required information in the S&C registration form. The AuthenticationManager then sends a request to verify company identity. If this succeeds, the system checks the email address and saves company information, finalizing the registration. Whether in the case of failed authentication or already used email, the system shows an error message to companies.

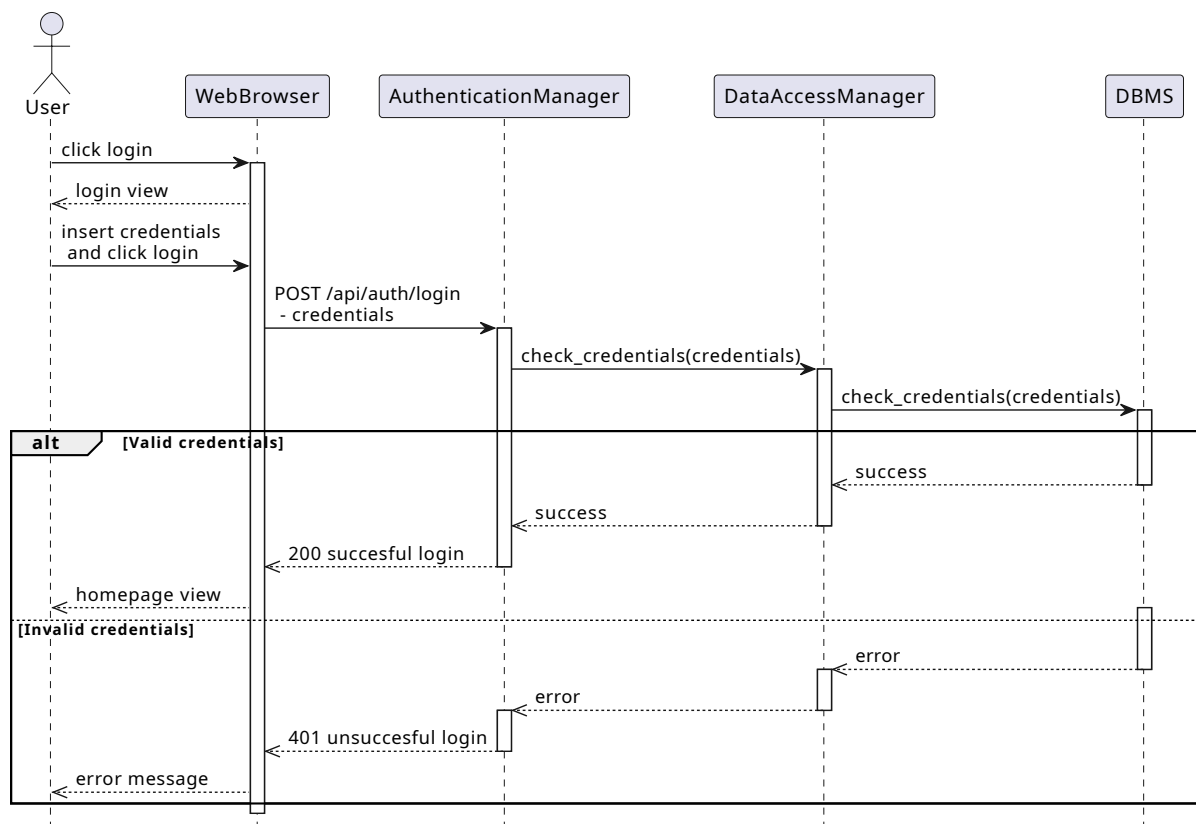
[UC3] - User login**Figure 2.6: User login Sequence Diagram**

Figure 2.6 shows user login. This procedure, handled by the AuthenticationManager, is common to both students and companies, since both sign in using their email address and password. The system checks the inserted credentials and provides back users with their homepage if they are correct. Otherwise, an error message is shown.

[UC4] - CV upload

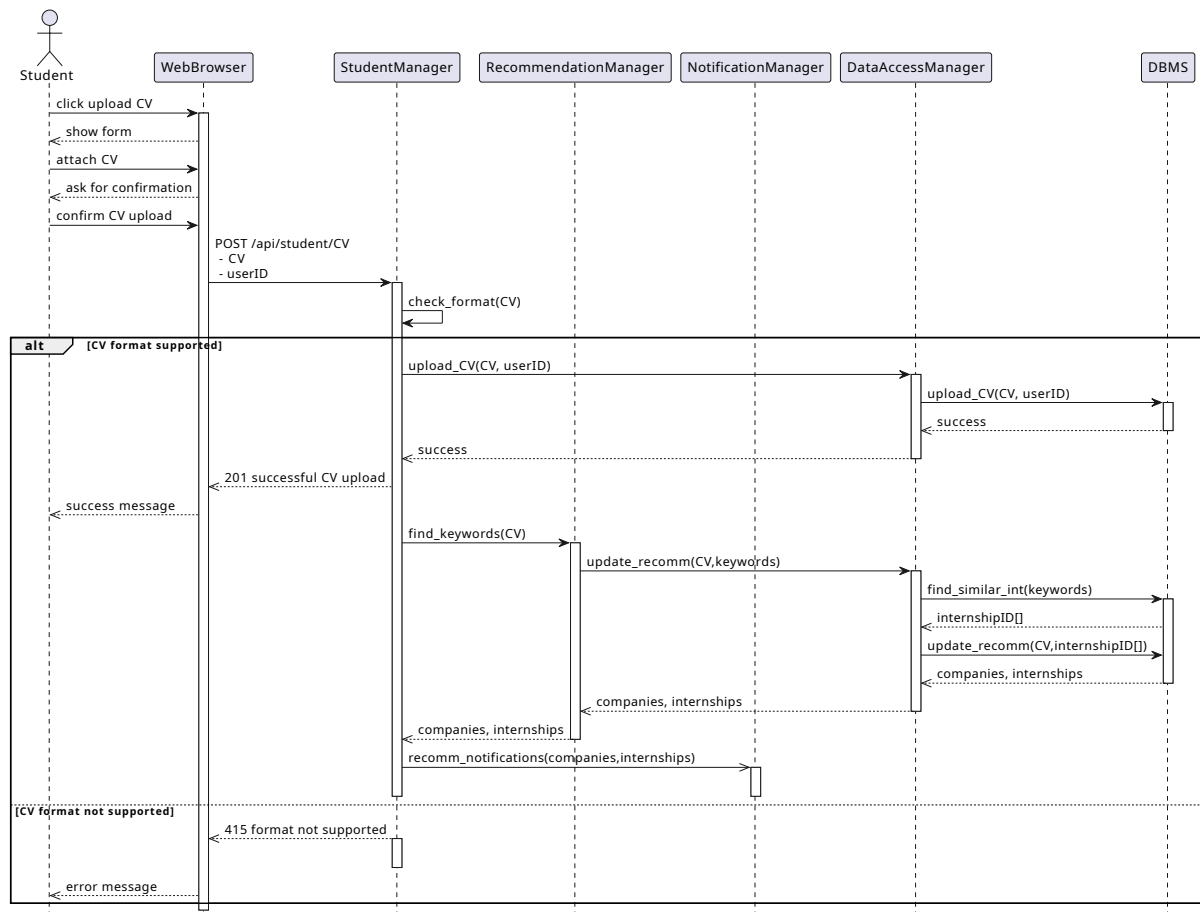


Figure 2.7: CV upload Sequence Diagram

Figure 2.7 shows the CV upload by a student. The StudentManager handles the request, checking the format of the submitted document, and delegating the DataAccesManager to save it in the database. After returning a successful response to the WebBrowser, a recommendation process starts. Firstly, the RecommendationManager computes the keywords of the uploaded CV. Then, through the DataAccessManager, recommendations for companies are updated. A series of queries, to find matching internships (exploiting MondoDB Atlas Search for full-text search) and set the new CV as recommended for them, are carried out. Companies and related internships are returned back to the StudentManager, that issues an asynchronous message for the NotificationManager, responsible for notifying companies of the new recommended CV.

[UC5] - CV removal

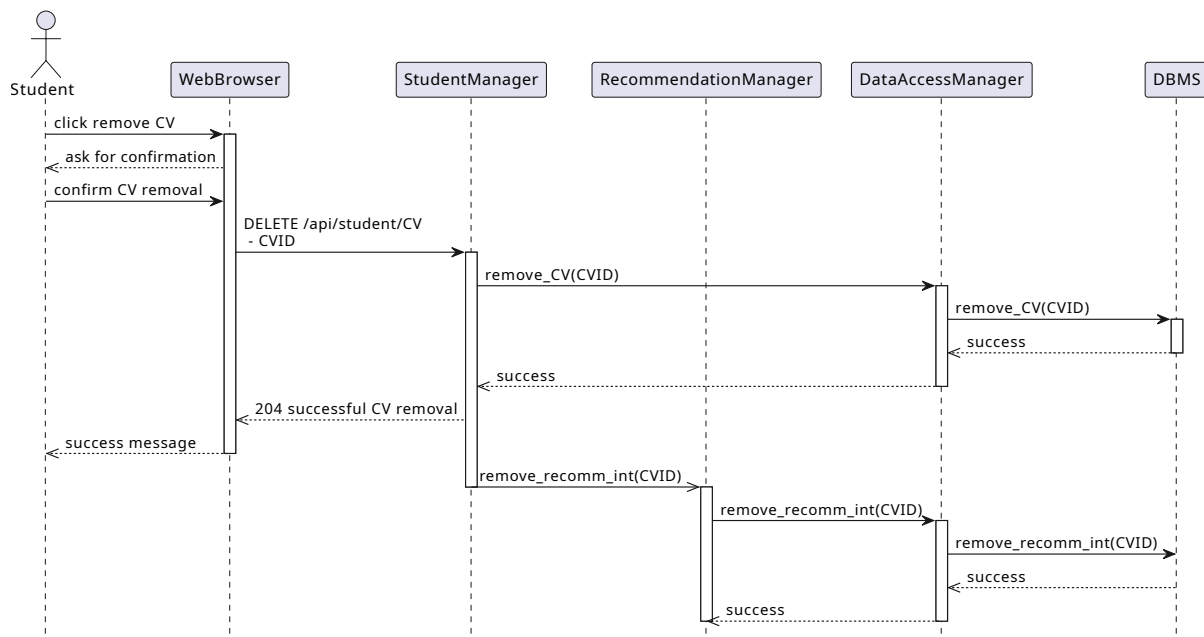


Figure 2.8: CV removal Sequence Diagram

Figure 2.8 shows the removal of a CV. As for all student operations, the StudentManager receives the request, and delegates the DataAccessManager to remove the student's CV. After returning a successful response to the client, an asynchronous call to the RecommendationManager is issued to remove the deleted CV from all recommendations in which it appeared.

[UC6] - Internship publication

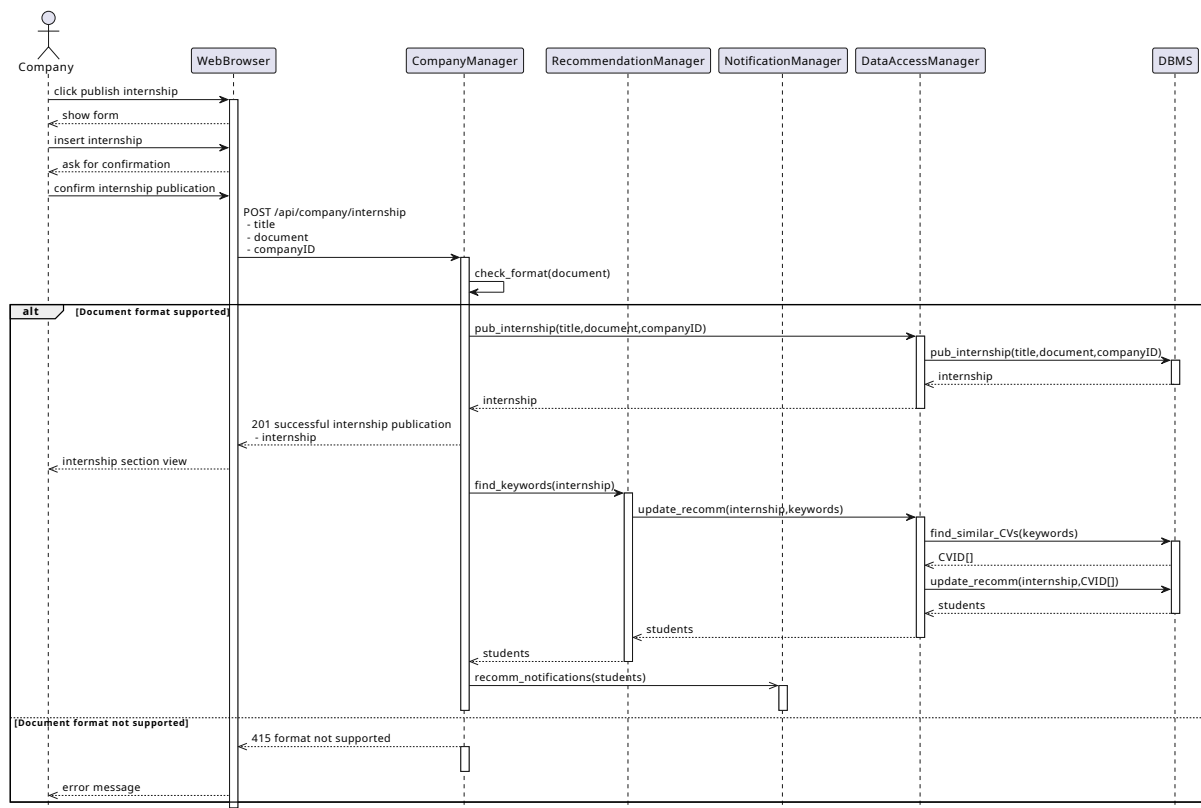


Figure 2.9: Internship publication Sequence Diagram

Figure 2.9 shows the internship publication by a company. The process is totally analogous to the CV upload of UC4. The CompanyManager handles the request, checks the format of the document and saves (through the DataAccessManager) the new internship. After sending back a response to the client, student recommendations are updated symmetrically to what is explained in UC4. Finally, students are notified of a new recommended internship.

[UC7] - Internship removal

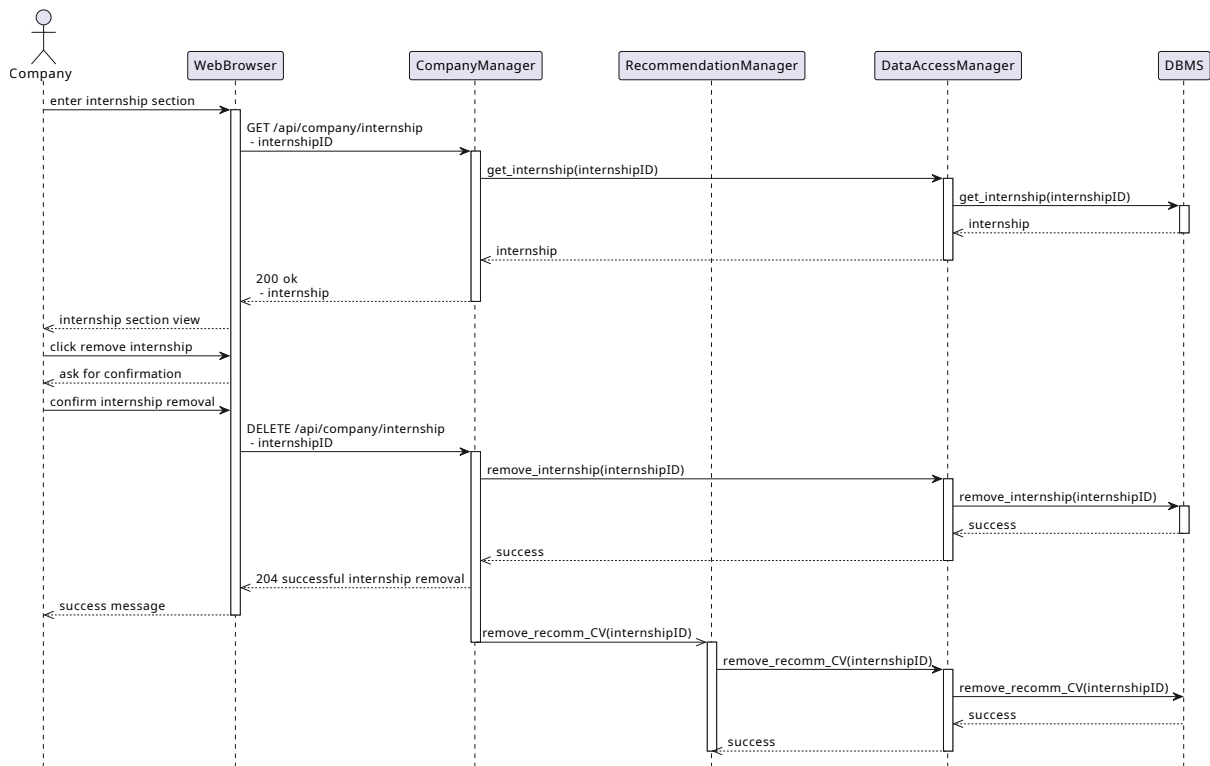


Figure 2.10: Internship removal Sequence Diagram

Figure 2.10 shows internship removal. This process is similar to the CV removal illustrated in UC5, but it differs from it since companies first need to access a specific internship section before removing it from the platform. A first HTTPS request, handled by the CompanyManager, is therefore needed to retrieve internship information. After that, the flow is the same as the one shown in UC5, except for the notification, which is not needed in this case: the internship is deleted from the database and it is removed from all CV recommendations in which it appeared.

[UC8] - CV selection

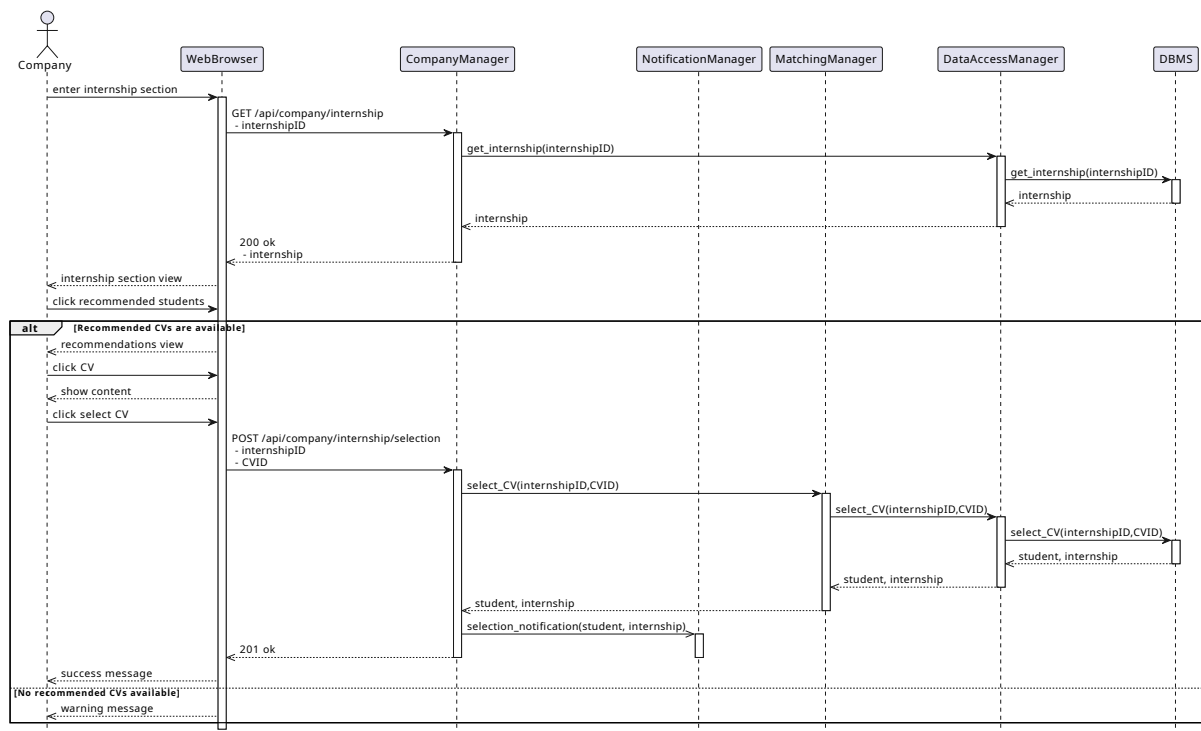


Figure 2.11: CV selection Sequence Diagram

Figure 2.11 shows the CV selection by a company. The process happens in the context of a specific internship, for which companies need to access the related section. The first workflow is therefore needed to retrieve its information, including recommended CVs. After that, the MatchingManager has the role of keeping track of the selection and saving it in the database through the DataAccessManager. Finally, the NotificationManager, triggered by the CompanyManager, informs the involved student, also letting them know for which internship they were selected. If for whatever reason, no recommendations are found, a warning message is shown to the company.

[UC9] - Internship selection

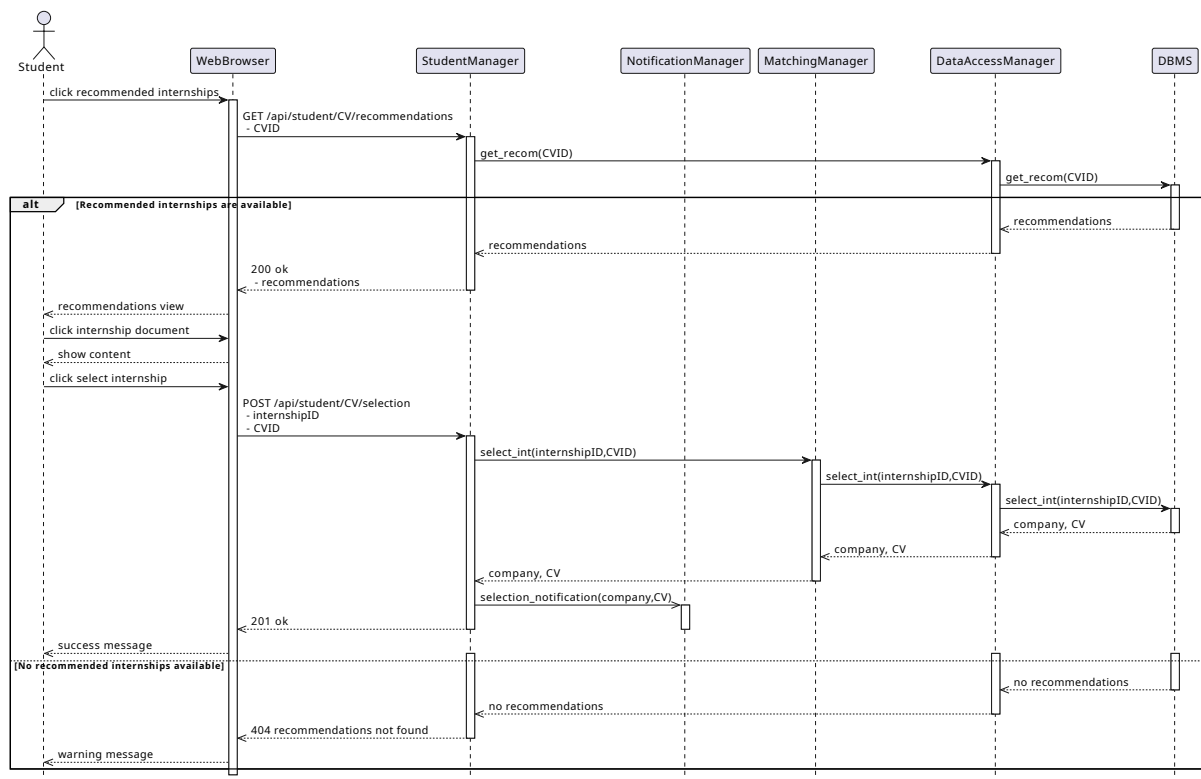


Figure 2.12: Internship selection Sequence Diagram

Figure 2.12 shows the internship selection by a student. Symmetrically to what is described in UC8, the StudentManager first retrieves internship recommendations for the student. After that, the MatchingManager stores the selection and the NotificationManager notifies the involved company that a student selected one of their internships. Also in this case, a warning message is displayed for students in case of missing recommendations.

[UC10] - Match creation by a student

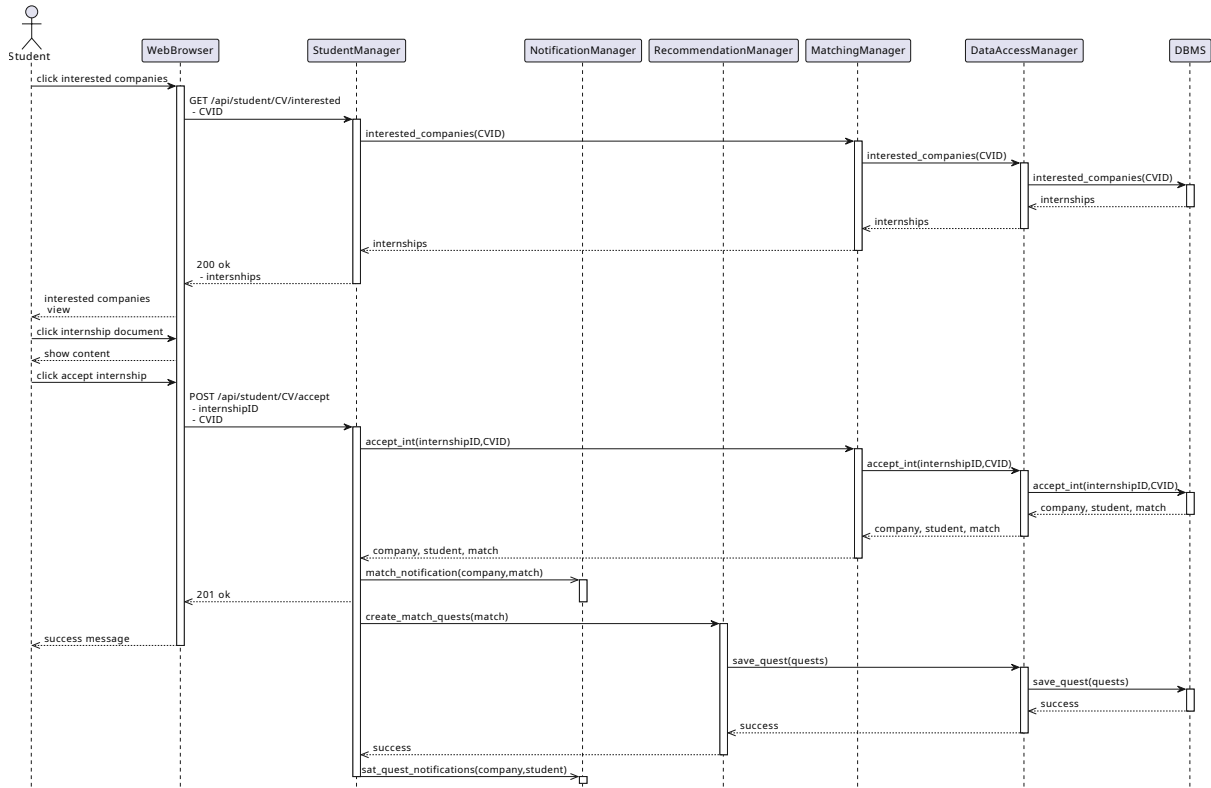


Figure 2.13: Match creation by a student Sequence Diagram

Figure 2.13 shows a match creation by a student. This happens after a company selected a student CV, in the case the student decides to accept the internship. Firstly, the list of interested companies (and related internships) is retrieved for the student, who then accepts one of them. The MatchingManager handles and creates the match, saving its information in the database through the DataAccessManager. After that, the StudentManager issues an asynchronous request for the NotificationManager, to inform the related company of the event. Once a successful response has been sent back to the student, the StudentManager triggers the RecommendationManager, that has the role of creating and saving satisfaction questionnaires for both involved student and company. As a reminder, these questionnaires are exploited by the system to evaluate matches and feed statistical analysis for future recommendations. After they have been correctly saved, a notification is sent to both parties to inform them of the available questionnaires.

[UC11] - Match creation by a company

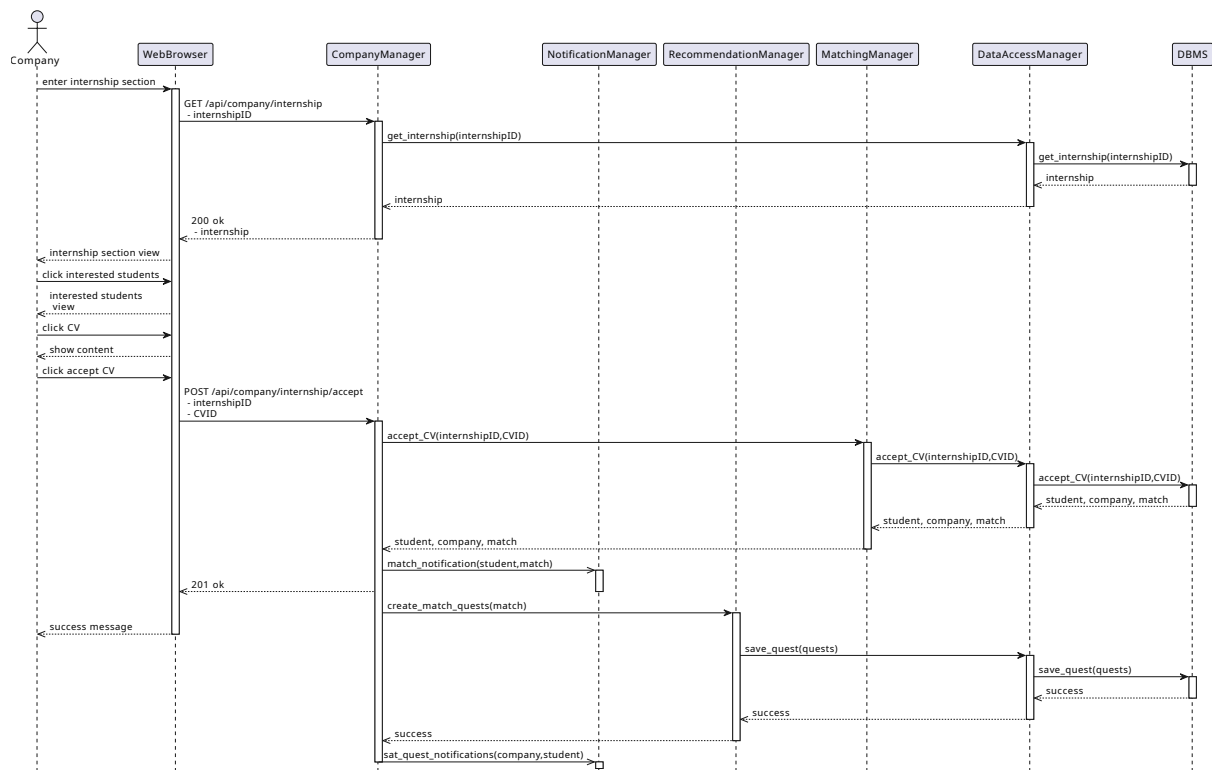


Figure 2.14: Match creation by a company Sequence Diagram

Figure 2.14 shows a match creation by a company. The process is totally analogous to the one described and illustrated in UC10. Firstly, internship information for the company to access its section is retrieved. Then, the match is formed, the student is notified, satisfaction questionnaires are created and their presence is communicated to both parties.

[UC12] - Interview set up

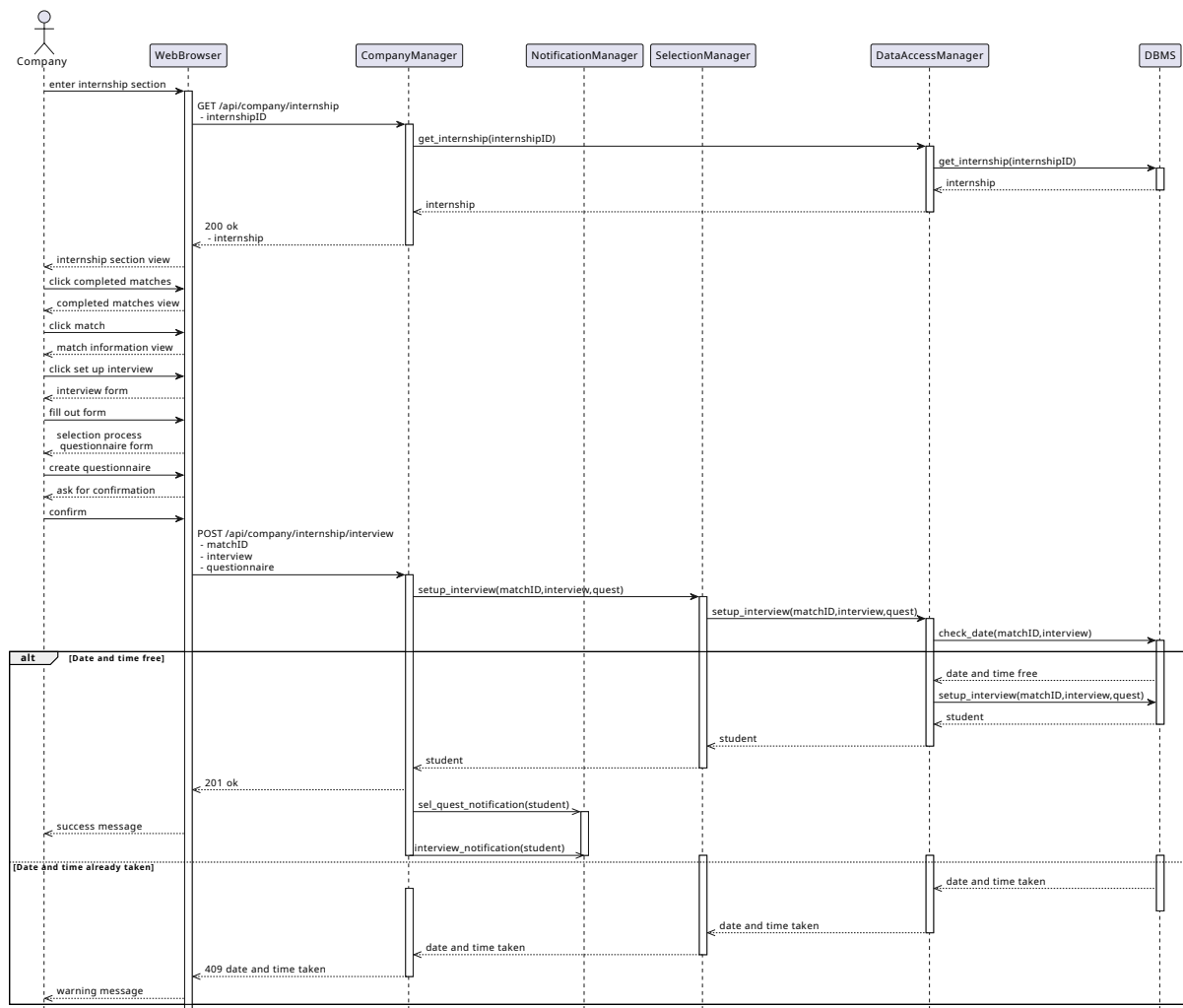


Figure 2.15: Interview set up Sequence Diagram

Figure 2.15 shows the interview set up for the selection process of an internship. During it, companies can enter an internship section, for which information is retrieved with the first HTTPS request, and set up an interview with a matching student. In addition to that, the company can also create a questionnaire to administer to students. The CompanyManager handles the request and delegates the SelectionManager to store interview and questionnaire information. In case the selected time slot for the interview is free, a successful response is returned to the company, and the involved student is notified by the NotificationManager of both new interview and questionnaire. Otherwise, in case the company already has a scheduled interview in the selected time slot, a warning message is displayed to it.

[UC13] - Selection process questionnaire compilation

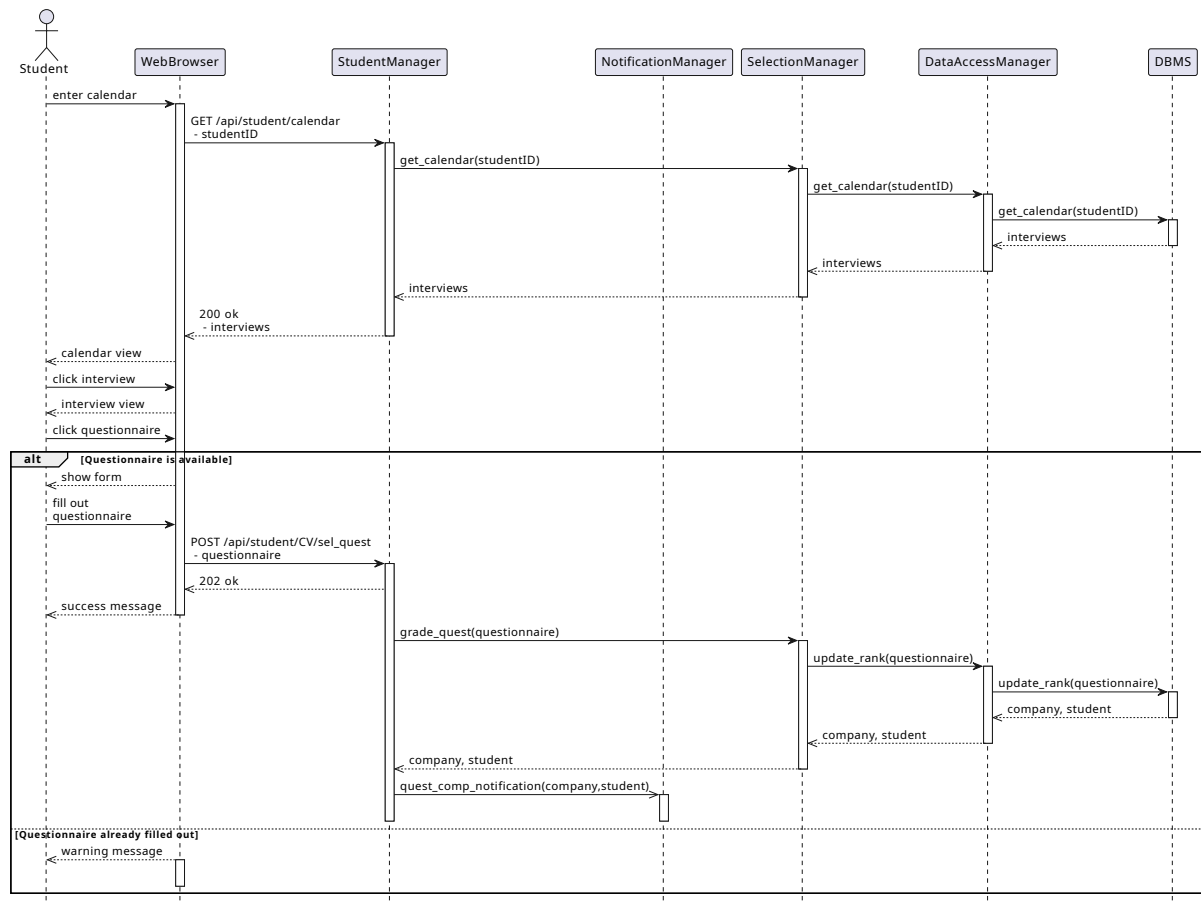


Figure 2.16: Selection process questionnaire compilation Sequence Diagram

Figure 2.16 shows student compilation of a selection process questionnaire. When a student accesses their calendar, a first request, handled by the StudentManager and delegated to the SelectionManager, is sent to retrieve information about scheduled interviews and related questionnaires. Then, after the student submits the filled out questionnaire, the SelectionManager grades it and updates the suitability rank for the related internship's selection process. Lastly, the involved company is notified by the NotificationManager of a new questionnaire compilation. In case of an already compiled questionnaire, a warning message is displayed to the student.

[UC14] - Student admission

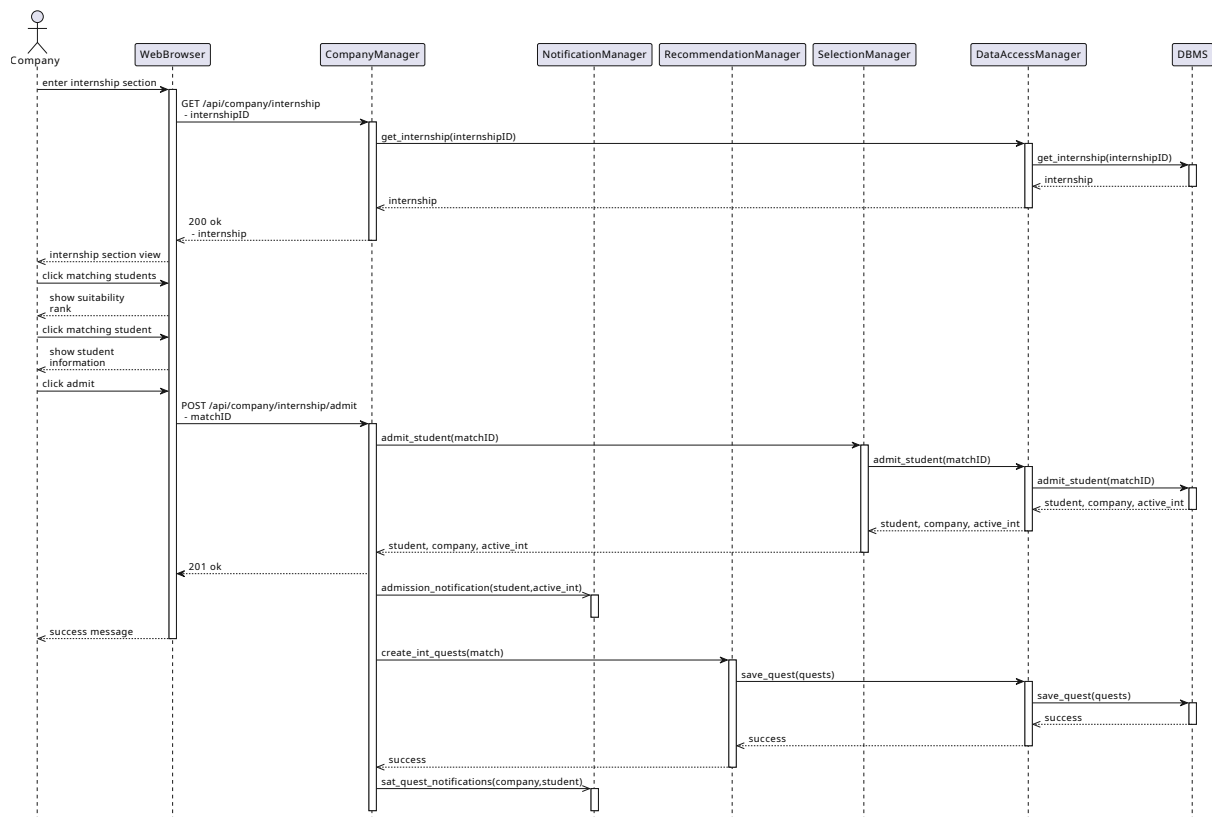


Figure 2.17: Student admission Sequence Diagram

Figure 2.17 shows the admission of a student for an internship. Firstly, information about the latter is retrieved for the company to enter its section. Then, a new request, handled by the CompanyManager and delegated to the SelectionManager, is needed to admit the student and update data about the new active internship. After a successful response has been sent back to the company, the admitted student is notified by the NotificationManager. Lastly, the RecommendationManager is triggered to create and save satisfaction questionnaires for the active internship, of which both involved student and company are notified.

[UC15] - Student rejection

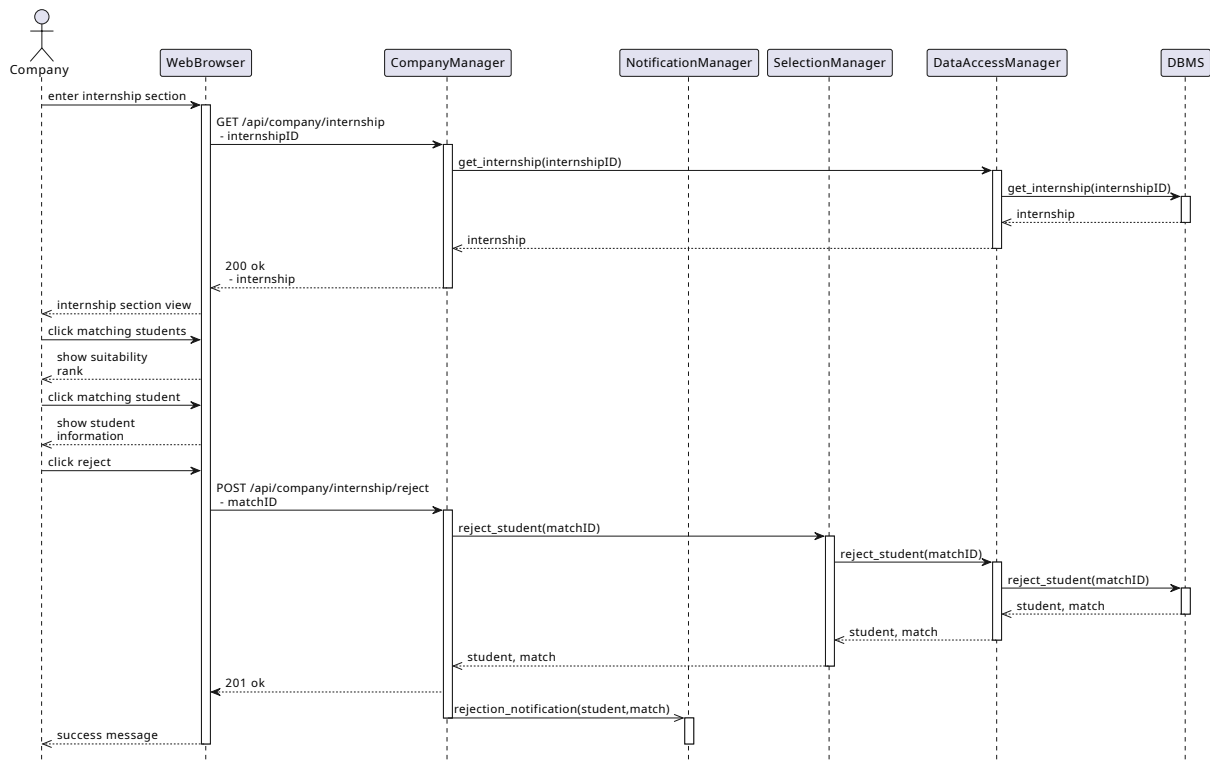


Figure 2.18: Student rejection Sequence Diagram

Figure 2.18 shows the rejection of a student for an internship. After internship information has been retrieved to let the company access its section, a request to reject a student is sent to the CompanyManager. The SelectionManager handles the rejection by updating data, through the DataAccessmanager, and finally the rejected student is notified by the NotificationManager.

[UC16] - Student satisfaction questionnaire compilation

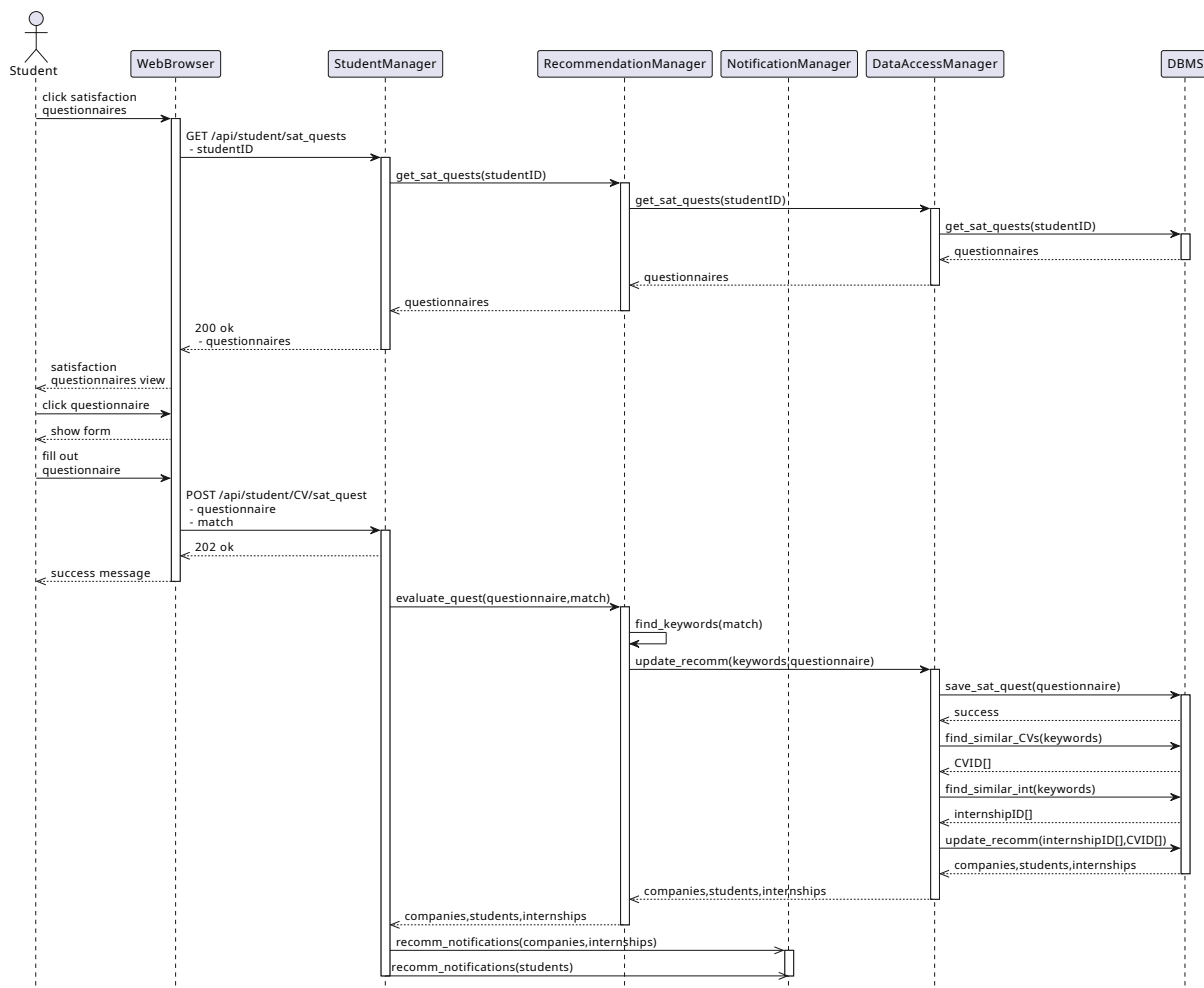


Figure 2.19: Student satisfaction questionnaire compilation Sequence Diagram

Figure 2.19 shows the compilation of a satisfaction questionnaire by a student. Firstly, the student enters the section dedicated to their satisfaction questionnaires, which are retrieved through the first request. After submitting the questionnaire, the student is acknowledged by the StudentManager, that delegates the process to the RecommendationManager. It first assigns a boolean evaluation (positive or negative) to the questionnaire, and consequently to the match between the student CV and the related internship. Then, it computes keywords from these documents to update recommendations in case of a positive evaluation. Through a series of queries, performed by the DataAccessManager, internships similar to the one for which the student has filled out the questionnaire, are recommended to similar students, in case of positively evaluated matches (i.e. questionnaires). Students and companies involved in new recommendations are finally returned to the StudentManager, that triggers the NotificationManager to inform the parties.

[UC17] - Company satisfaction questionnaire compilation

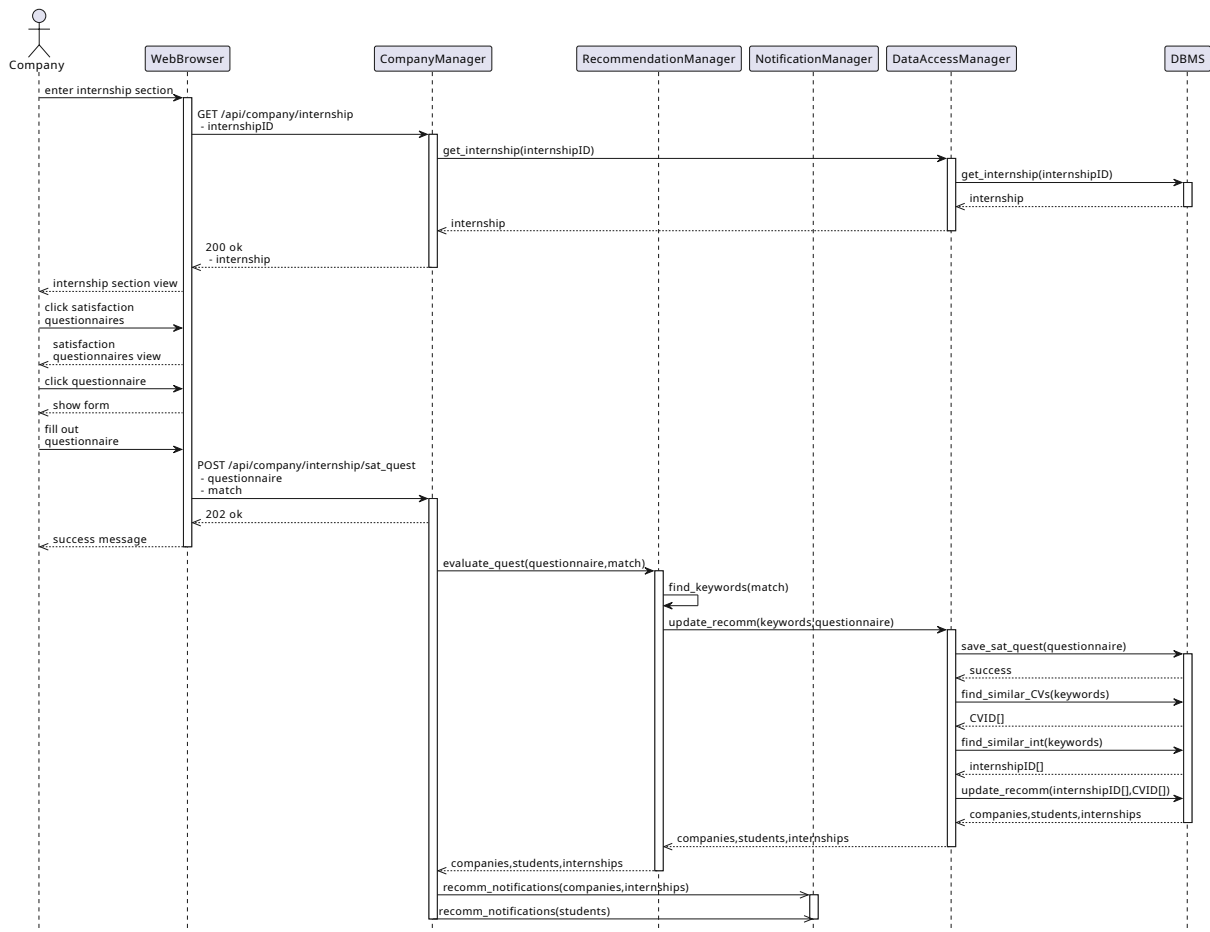


Figure 2.20: Company satisfaction questionnaire compilation Sequence Diagram

Figure 2.20 shows the compilation of a satisfaction questionnaire by a company. The process is totally analogous to the one described in UC16. The only difference is that the company needs to access a specific internship section for which a questionnaire is available. After submitting it, the workflow is symmetric to UC16: the questionnaire is evaluated, new recommendations are computed and involved parties are notified.

[UC18] - Student message providing

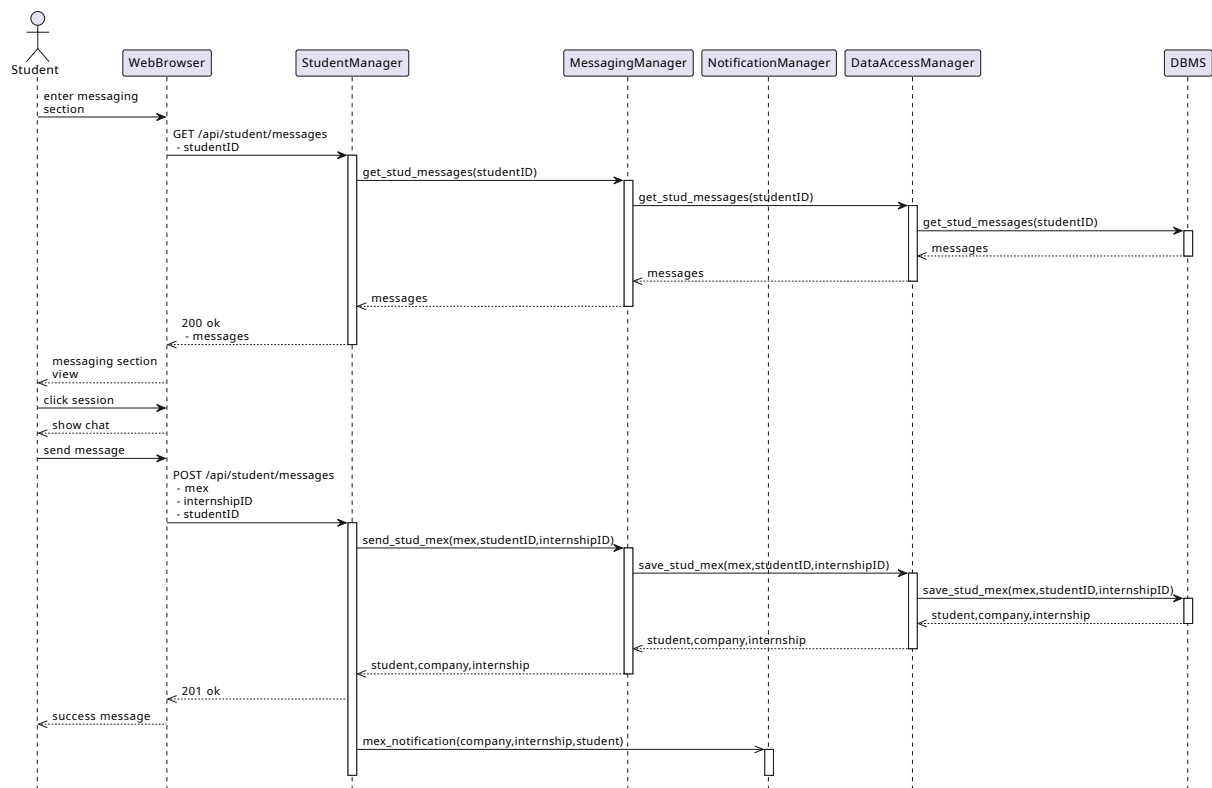


Figure 2.21: Student message providing Sequence Diagram

Figure 2.21 shows the sending of a message by a student. For each active internship, a chat exists between the involved student and company. The MessagingManager, triggered by the StudentManager, totally handles the process, retrieving messages during the first interaction, and actually saving the sent message during the second one. Finally, after the student has been acknowledged of the sending, the corresponding company is notified by the NotificationManager.

[UC19] - Company message providing

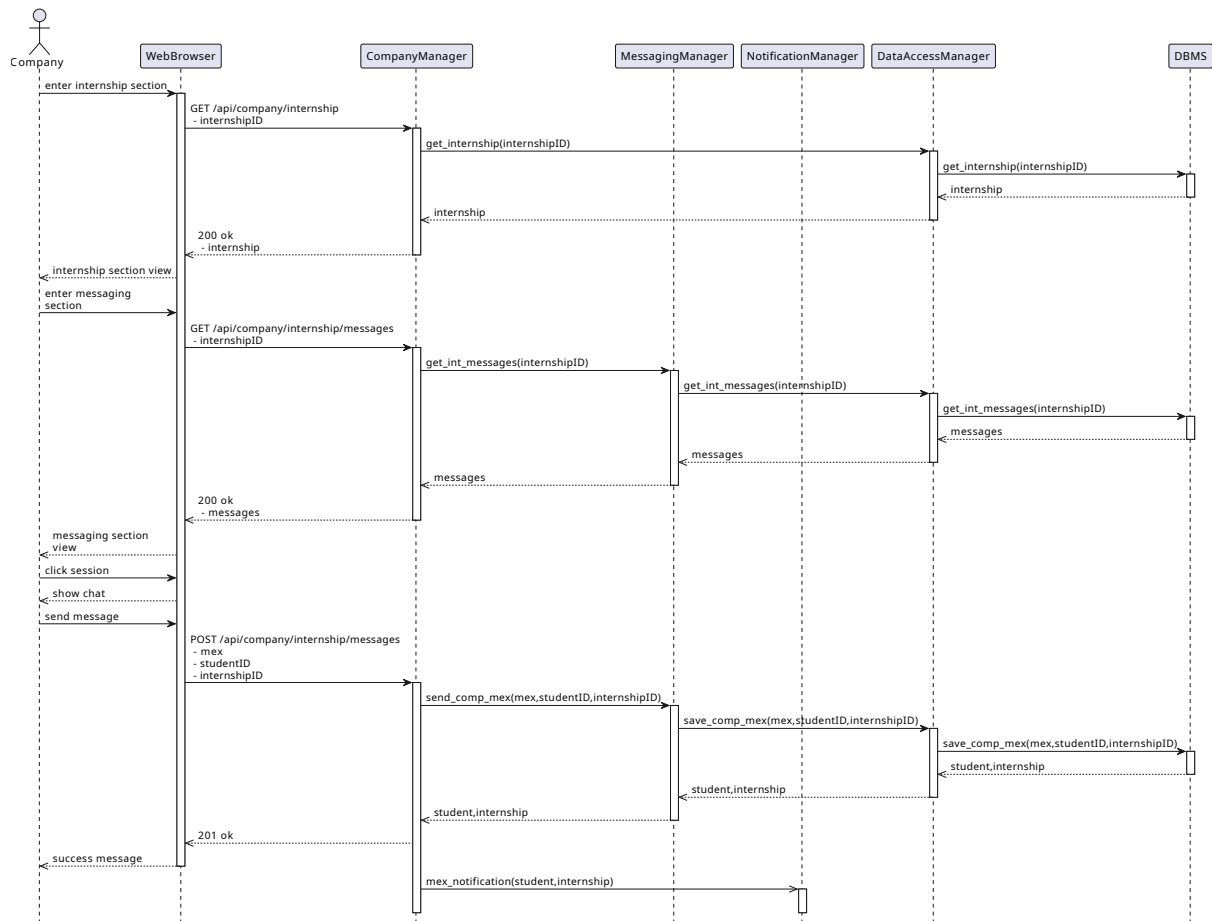


Figure 2.22: Company message providing Sequence Diagram

Figure 2.22 shows the sending of a message by a company. The only difference from UC18, is that companies have a messaging section for each of their internships, and therefore first need to access an internship section. After that, the workflow is the same: the MessagingManager retrieves chats and stores the new message in the two requests. Finally, the NotificationManager, notifies the involved student.

2.5. Component Interfaces

In this section, component interfaces included in the sequence diagrams of the runtime view are described in detail.

For RESTful APIs, parameters and return objects types won't be specified, as they will all be encoded into JSON structures according to REST standards and following the application server logic. Also, note that error responses caused by server errors (denoted by 5xx status code), have not been considered for the sake of simplicity.

2.5.1. Application Server - exposed API

AuthenticationManagerInterface

Request	GET /api/auth/start_stud_ver
Parameters	student information
Description	Requests the student verification and redirects the student to the verification platform as a response (status code 302).

Table 2.1: Authentication Manager REST request - start student verification

Request	GET /api/auth/verify_student
Parameters	institutional credentials
Description	Hands over the student enrollment verification to the Student Verification Platform. Redirects the student to the registration page (status code 302), providing a success parameter to the Web Browser in the case of successful verification or an error parameter in the case of failed verification.

Table 2.2: Authentication Manager REST request - student verification

Request	POST /api/auth/reg_student
Parameters	student information
Description	Hands over the student registration to the Data Access Manager. Provides a success response to the Web Browser (status code 201) if the inserted e-mail address is unique, or an error response (status code 409) if the address is already used.

Table 2.3: Authentication Manager REST request - student registration

Request	POST /api/auth/reg_company
Parameters	company information
Description	Handles the company registration process. Provides a success response to the Web Browser (status code 201) if the inserted e-mail address is unique and an error response if the address is already used (status code 409) or in the case of failed company verification (status code 404).

Table 2.4: Authentication Manager REST request - company registration

Request	POST /api/auth/login
Parameters	credentials
Description	Hands over user login to the Data Access Manager. If the inserted credentials are valid, it provides a success response to the Web Browser (status code 200), otherwise it provides an error response (status code 401).

Table 2.5: Authentication Manager REST request - user login

StudentManagerInterface

Request	POST /api/student/CV
Parameters	CV, userID
Description	Handles the CV upload process by interacting with the Data Access Manager. Returns a success response to the Web Browser in the case of successful upload (status code 201) or an error response if the CV document format is not supported (status code 415).

Table 2.6: Student Manager REST request - CV upload

Request	DELETE /api/student/CV
Parameters	CVID
Description	Handles the CV removal process by interacting with the Data Access Manager. Returns a success response to the Web Browser (status code 204).

Table 2.7: Student Manager REST request - CV removal

Request	GET /api/student/CV/recommendations
Parameters	CVID
Description	Retrieves the set of recommended internships for a specific student by interacting with the Data Access Manager. If there are any recommendations, it returns them to the Web Browser (status code 201), otherwise it returns a error response (status code 404).

Table 2.8: Student Manager REST request - recommended internships retrieval

Request	POST /api/student/CV/selection
Parameters	internshipID, CVID
Description	Hands over the internship selection process to the Matching Manager. Returns a success response to the Web Browser (status code 201).

Table 2.9: Student Manager REST request - internship selection

Request	GET /api/student/CV/interested
Parameters	CVID
Description	By interacting with the Matching Manager, it retrieves the set of internships for which companies have selected the student's CV and returns them to the Web Browser (status code 200).

Table 2.10: Student Manager REST request - interested companies retrieval

Request	POST /api/student/CV/accept
Parameters	internshipID, CVID
Description	Hands over the internship acceptance process to the Matching Manager. Returns a success response to the Web Browser (status code 201).

Table 2.11: Student Manager REST request - match creation

Request	GET /api/student/calendar
Parameters	studentID
Description	Retrieves the set of planned interviews for a student by interacting with the Selection Manager. Returns interviews information to the Web Browser (status code 200).

Table 2.12: Student Manager REST request - calendar retrieval

Request	POST /api/student/CV/sel_quest
Parameters	questionnaire
Description	Hands over the evaluation of a selection process questionnaire to the Selection Manager and returns a success response to the Web Browser (status code 202).

Table 2.13: Student Manager REST request - selection questionnaire compilation

Request	GET /api/student/sat_questions
Parameters	studentID
Description	Retrieves the set of student satisfaction questionnaires by interacting with the Recommendation Manager. Returns the questionnaires to the Web Browser (status code 200).

Table 2.14: Student Manager REST request - satisfaction questionnaires retrieval

Request	POST /api/student/CV/sat_quest
Parameters	questionnaire, match
Description	Hands over the evaluation of a student satisfaction questionnaire to the Recommendation Manager and returns a success response to the Web Browser (status code 202).

Table 2.15: Student Manager REST request - satisfaction questionnaire compilation

Request	GET /api/student/messages
Parameters	studentID
Description	Retrieves the messaging sessions related to student's active internships by interacting with the Messaging Manager. Returns the messages to the Web Browser (status code 200).

Table 2.16: Student Manager REST request - messages retrieval

Request	POST /api/student/messages
Parameters	mex, internshipID, studentID
Description	After the student sends a new message to a company, it hands over the process of messaging session update to the Messaging Manager. Returns a success response to the Web Browser (status code 201).

Table 2.17: Student Manager REST request - message communication

CompanyManagerInterface

Request	POST /api/company/internship
Parameters	title, document, companyID
Description	Handles the internship publication process by interacting with the Data Access Manager. Returns a success response to the Web Browser in the case of successful publication (status code 201) or an error response if the internship document format is not supported (status code 415).

Table 2.18: Company Manager REST request - internship publication

Request	GET /api/company/internship
Parameters	internshipID
Description	By interacting with the Data Access Manager, it retrieves all information related to a published internship (recommended CVs, interested students, completed matches...). Returns the obtained data to the Web Browser (status code 200).

Table 2.19: Company Manager REST request - internship section retrieval

Request	DELETE /api/company/internship
Parameters	internshipID
Description	Handles the internship removal process by interacting with the Data Access Manager. Returns a success response to the Web Browser (status code 204).

Table 2.20: Company Manager REST request - internship removal

Request	POST /api/company/internship/selection
Parameters	internshipID, CVID
Description	Hands over the CV selection process to the Matching Manager. Returns a success response to the Web Browser (status code 201).

Table 2.21: Company Manager REST request - CV selection

Request	POST /api/company/internship/accept
Parameters	internshipID, CVID
Description	Hands over the CV acceptance process to the Matching Manager. Returns a success response to the Web Browser (status code 201).

Table 2.22: Company Manager REST request - match creation

Request	POST /api/company/internship/interview
Parameters	matchID, interview, questionnaire
Description	Hands over the interview set up process to the Selection Manager. If the specified date and time are free, it returns a success response to the Web Browser (status code 201), otherwise it returns an error response (status code 409).

Table 2.23: Company Manager REST request - interview set up

Request	POST /api/company/internship/admit
Parameters	matchID
Description	Hands over the admission of a student to the Selection Manager. Returns a success response to the Web Browser (status code 201).

Table 2.24: Company Manager REST request - student admission

Request	POST /api/company/internship/reject
Parameters	matchID
Description	Hands over the rejection of a student to the Selection Manager. Returns a success response to the Web Browser (status code 201).

Table 2.25: Company Manager REST request - student rejection

Request	POST /api/company/internship/sat_quest
Parameters	questionnaire, match
Description	Hands over the evaluation of a company satisfaction questionnaire to the Recommendation Manager and returns a success response to the Web Browser (status code 202).

Table 2.26: Company Manager REST request - satisfaction questionnaire compilation

Request	GET /api/company/internship/messages
Parameters	internshipID
Description	Retrieves the messaging sessions related to an internship by interacting with the Messaging Manager. Returns the messages to the Web Browser (status code 200).

Table 2.27: Company Manager REST request - messages retrieval

Request	POST /api/company/internship/messages
Parameters	mex, studentID, internshipID
Description	After a company sends a new message to a student, it hands over the process of messaging session update to the Messaging Manager. Returns a success response to the Web Browser (status code 201).

Table 2.28: Company Manager REST request - message communication

2.5.2. External APIs

StudentVerificationPlatformInterface

Request	GET /stud_ver_api/start_verification
Parameters	university
Description	Sets up the enrollment verification using student's university information. Returns a success response to the Authentication Manager (status code 200).

Table 2.29: Student Verification Platform REST request - redirect to platform

Request	GET /stud_ver_api/verify
Parameters	institutional credentials
Description	Verifies if a student is correctly enrolled in the specified university through their institutional credentials. Returns a success response to the Authentication Manager in the case of successful verification (status code 200) or an error message in the case of failed verification (status code 404).

Table 2.30: Student Verification Platform REST request - student verification

CompanyVerificationPlatformInterface

Request	GET /comp_ver_api/verify
Parameters	business registration number
Description	Verifies the identity of a company through its business registration number. Returns a success response to the Authentication Manager in the case of successful verification (status code 200) or an error message in the case of failed verification (status code 404).

Table 2.31: Company Verification Platform REST request - company verification

2.5.3. Application Server - internal interfaces

RecommendationManager

- `find_keywords(CV CV)`

Extracts keywords from a CV document and hands over recommendations update to the Data Access Manager. Returns an *Object* containing new recommendations (internships and related companies) for the corresponding student.

- `remove_recomm_int(Integer CVID)`

By interacting with the Data Access Manager, removes from all internships for which the canceled CV has been recommended all the related recommendations.

- `find_keywords(Internship internship)`

Extracts keywords from an internship document and hands over recommendations update to the Data Access Manager. Returns an *Object* containing new recommendations (students) for the corresponding company.

- `remove_recomm_CV(Integer internshipID)`

By interacting with the Data Access Manager, removes from all CVs for which the canceled internship has been recommended all the related recommendations.

- `create_match_requests(Match match)`

Creates two satisfaction questionnaires for a completed matchmaking process (one for the student and one for the company).

- `create_int_requests(Match match)`

Creates two satisfaction questionnaires for an activated internship (one for the student and one for the company).

- `get_sat_requests(Integer studentID)`

Retrieves the set of satisfaction questionnaires for a student by interacting with the Data Access Manager. Returns a *List<Questionnaire>* object to the Student Manager.

- `evaluate_quest(Questionnaire questionnaire, Match match)`

Evaluates a satisfaction questionnaire for a matchmaking process or an active internship. Returns an *Object* containing information about students and internships

(and related companies) whose recommendations have been updated after the questionnaire evaluation.

MatchingManagerInterface

- `select_CV(Integer CVID, Integer internshipID)`

Hands over the CV selection process for the provided internship to the Data Access Manager. Returns an *Object* containing information about related student and internship.

- `select_int(Integer internshipID, Integer CVID)`

Hands over the internship selection process to the Data Access Manager. Returns an *Object* containing information about related company and CV.

- `interested_companies(Integer CVID)`

By interacting with the Data Access Manager, it retrieves the set of internships for which companies have selected the student's CV. Returns a *List<Internship>* object to the Student Manager.

- `accept_int(Integer internshipID, Integer CVID)`

Hands over the internship acceptance process to the Data Access Manager. Returns an *Object* containing information about the created match and the related company and student.

- `accept_CV(Integer internshipID, Integer CVID)`

Hands over the CV acceptance process for the provided internship to the Data Access Manager. Returns an *Object* containing information about the created match and the related company and student.

SelectionManagerInterface

- `setup_interview(Integer matchID, Interview interview, Questionnaire quest)`

Hands over the process of interview set up to the Data Access Manager. If the specified date and time are free, it returns a *Student* object (related to the interviewed student) to the Company Manager, otherwise it throws an *Exception*.

- `get_calendar(Integer studentID)`

Retrieves the set of planned interviews for a student by interacting with the Data Access Manager. Returns a *List<Interview>* object to the Student Manager.

- `grade_quest(Questionnaire questionnaire)`

Grades a selection process questionnaire filled out by a student and hands over the suitability rank update (for the company that created the questionnaire) to the Data Access Manager. Returns an *Object* containing information about the involved student and company.

- `admit_student(Integer matchID)`

Hands over the admission of a student to the Data Access Manager. Returns an *Object* containing information about the activated internship and the involved student and company.

- `reject_student(Integer matchID)`

Hands over the rejection of a student to the Data Access Manager. Returns an *Object* containing information about the related match and the involved student.

MessagingManagerInterface

- `get_stud_messages(Integer studentID)`

Retrieves the messaging sessions related to student's active internships by interacting with the Data Access Manager. Returns a *List<Message>* object to the Student Manager.

- `send_stud_mex(Message mex, Integer studentID, Integer internshipID)`

After the student sends a new message to a company, it hands over the process of messaging session update to the Data Access Manager. Returns an *Object* containing information about the related active internship and the involved student and company.

- `get_int_messages(Integer internshipID)`

Retrieves the messaging sessions related to an internship by interacting with the Messaging Manager. Returns a *List<Message>* to the Company Manager.

- `send_comp_mex(Message mex, Integer studentID, Integer internshipID)`

After the company sends a new message to a student, it hands over the process of messaging session update to the Data Access Manager. Returns an *Object* containing information about the related active internship and the involved student and company.

NotificationManagerInterface

- `recomm_notifications(List<Company> companies, List<Internship> internships)`
Notifies the provided companies of new recommended CVs for some of their internships.
- `recomm_notifications(List<Student> students)`
Notifies the provided students of new recommended internships.
- `selection_notification(Student student, Internship internship)`
Notifies the provided student that their CV has been selected for the provided internship.
- `selection_notification(Company company, CV CV)`
Notifies the provided company that a student has selected one of their internships.
- `match_notification(Company company, Match match)`
Notifies the provided company of a created match between one of its internships and a student.
- `sat_quest_notifications(Company company, Student student)`
Notifies company and student involved in a match about the availability of a satisfaction questionnaire.
- `match_notification(Student student, Match match)`
Notifies the provided student of a created match between their CV and an internship.
- `sel_quest_notification(Student student)`
Notifies the provided student about the availability of a selection process questionnaire.
- `interview_notification(Student student)`
Notifies the provided student about a new scheduled interview.
- `quest_comp_notification(Company company, Student student)`
Notifies the provided company that a student has compiled a questionnaire for the selection process regarding one of its internships.
- `admission_notification(Student student, Match active_int)`

Notifies the provided student about their admission to a matching internship.

- `rejection_notification(Student student, Match match)`

Notifies the provided student about their rejection for a matching internship.

- `mex_notification(Company company, Internship internship, Student student)`

Notifies the provided company about a message sent by the provided student in the context of an active internship.

- `mex_notification(Student student, Internship internship)`

Notifies the provided student of a message sent by a company in the context of an active internship.

DataAccessManagerInterface

- `reg_student(List<String> student_info)`

Triggers queries to the DBMS in order to register a student through the provided information. If the student e-mail is unique, it returns a success message (*String*) to the Authentication Manager, otherwise it throws an *Exception*.

- `reg_company(List<String> company_info)`

Triggers queries to the DBMS in order to register a company through the provided information. If the company e-mail is unique, it returns a success message (*String*) to the Authentication Manager, otherwise it throws an *Exception*.

- `check_credentials(List<String> credentials)`

Triggers a query to the DBMS in order to check the existence of the provided user credentials. If credentials are valid, it returns a success message (*String*) to the Authentication Manager, otherwise it throws an *Exception*.

- `upload_CV(CV CV, Integer userID)`

Triggers a query to the DBMS in order to upload a student CV. Returns a success message (*String*) to the Student Manager.

- `update_recomm(CV CV, List<String> keywords)`

Triggers queries to the DBMS in order to update recommendations (for the owner of the provided CV) with the given keywords. Returns an *Object* containing new recommendations (internships and related companies) for the corresponding student.

- `remove_CV(Integer CVID)`

Triggers a query to the DBMS in order to delete a student CV. Returns a success message (*String*) to the Student Manager.

- `remove_recomm_int(Integer CVID)`

Triggers a query to the DBMS in order to delete all recommendations associated to the specified CV. Returns a success message (*String*) to the Recommendation Manager.

- `pub_internship(String title, Internship document, Integer companyID)`

Triggers a query to the DBMS in order to upload an internship. Returns an *Internship* object to the Company Manager.

- `update_recomm(Internship internship, List<String> keywords)`

Triggers queries to the DBMS in order to update recommendations (for the publisher of the provided internship) with the given keywords. Returns an *Object* containing new recommendations (students) for the corresponding company.

- `get_internship(Integer internshipID)`

Triggers a query to the DBMS in order to retrieve all information related to a published internship (recommended CVs, interested students, completed matches...). Returns an *Object* containing the obtained data to the Company Manager.

- `remove_internship(Integer internshipID)`

Triggers a query to the DBMS in order to delete an internship. Returns a success message (*String*) to the Company Manager.

- `remove_recomm_CV(Integer internshipID)`

Triggers a query to the DBMS in order to delete all recommendations associated to the specified internship. Returns a success message (*String*) to the Recommendation Manager.

- `select_CV(Integer internshipID, Integer CVID)`

Triggers a query to the DBMS in order to add the specified CV to the "selected CVs" for a given internship. Returns an *Object* containing information about the related student and internship.

- `get_recom(Integer CVID)`

Triggers a query to the DBMS in order to retrieve all recommended internships for a given student (identified by the provided CVID). If there are any recommendations, it returns a *List<Internship>* object to the Student Manager, otherwise it throws an *Exception*.

- *select_int(Integer internshipID, Integer CVID)*

Triggers a query to the DBMS in order to add the specified internship to the "selected internships" for a given student (identified by the provided CVID). Returns an *Object* containing information about the related company and CV.

- *interested_companies(Integer CVID)*

Triggers a query to the DBMS in order to retrieve all internships for which companies have selected the provided CV. Returns a *List<Internship>* object to the Matching Manager.

- *accept_int(Integer internshipID, Integer CVID)*

Triggers a query to the DBMS in order to create a match between the specified CV and internship. Returns an *Object* containing information about the created match and the related company and student.

- *save_quest(List<Questionnaire> quests)*

Triggers a query to the DBMS in order to save the provided questionnaires. Returns a success message (*String*).

- *accept_CV(Integer internshipID, Integer CVID)*

Triggers a query to the DBMS in order to create a match between the specified CV and internship. Returns an *Object* containing information about the created match and the related company and student.

- *setup_interview(Integer matchID, Interview interview, Questionnaire quest)*

Triggers queries to the DBMS in order to correctly set up an interview with a student in the context of a match. If the specified date and time are free, it returns a *Student* object (related to the interviewed student) to the Selection Manager, otherwise it throws an *Exception*.

- *get_calendar(Integer studentID)*

Triggers a query to the DBMS in order to retrieve the set of scheduled interviews for the provided student. Returns a *List<Interview>* object to the Selection Manager.

- `update_rank(Questionnaire questionnaire)`

Triggers a query to the DBMS in order to update the suitability rank for the company that created the provided selection process questionnaire. Returns an *Object* containing information about student and company involved in the selection process.

- `admit_student(Integer matchID)`

Triggers a query to the DBMS in order to activate an internship program based on the provided match. Returns an *Object* containing the information about the activated internship and the involved student and company.

- `reject_student(Integer matchID)`

Triggers a query to the DBMS in order to reject a student for an internship program based on the provided match. Returns an *Object* containing information about the related match and the involved student.

- `get_sat_requests(Integer studentID)`

Triggers a query to the DBMS in order to retrieve the set of satisfaction questionnaires for the provided student. Returns a *List<Questionnaire>* object to the Recommendation Manager.

- `update_recomm(List<String> keywords, Questionnaire questionnaire)`

Triggers queries to the DBMS in order to update recommendations for internships and CVs that match the provided keywords. Recommendations are updated or not based on the questionnaire evaluation. The interface returns an *Object* containing information about students and internships (and related companies) whose recommendations have been modified.

- `get_stud_messages(Integer studentID)`

Triggers a query to the DBMS in order to retrieve the messaging sessions related to the provided student. Returns a *List<Message>* object to the Messaging Manager.

- `save_stud_mex(Message mex, Integer studentID, Integer internshipID)`

Triggers a query to the DBMS in order to update a messaging session with the provided student message. Returns an *Object* containing information about the related active internship and the involved student and company.

- `get_int_messages(Integer internshipID)`

Triggers a query to the DBMS in order to retrieve the messaging sessions related to the provided internship. Returns a *List<Message>* object to the Messaging Manager.

- `save_comp_mex(Message mex, Integer studentID, Integer internshipID)`

Triggers a query to the DBMS in order to update a messaging session with the provided company message. Returns an *Object* containing information about the related active internship and the involved student.

2.5.4. Data Layer interfaces

As mentioned in Section 2.4, interactions between `DataAccessManager` and DBMS are represented as simple method invocations, whereas they involve complex MongoDB query executions. Therefore, parameter types won't be specified for Data Layer interfaces, as they will all be converted into JSON structures in order to comply with MongoDB standards and following the Application Server logic.

DBMSInterface

- `check_email(email)`

Finds the student or company associated to the provided e-mail address.

- `reg_student(student_info)`

Registers a new student by saving the provided information in the database.

- `reg_company(company_info)`

Registers a new company by saving the provided information in the database.

- `check_credentials(credentials)`

Finds the student or company associated to the provided user credentials.

- `upload_CV(CV, userID)`

Saves a new CV in the database and associates it to the student corresponding to the provided identifier.

- `find_similar_int(keywords)`

Finds internship documents that match the provided keywords.

- `update_recomm(CV, internshipID[ ])`

Updates recommendations for the provided CV with the given internships (and vice versa) and returns information about the latter and related companies.

- `remove_CV(CVID)`

Deletes the CV corresponding to the provided identifier from the database.

- `remove_recomm_int(CVID)`

Removes the CV corresponding to the provided identifier from all internships for which the first is recommended.

- `pub_internship(title, document, companyID)`

Saves a new internship in the database and associates it to the company corresponding to the provided identifier. Returns information about the the stored internship.

- `find_similar_CVs(keywords)`

Finds CV documents that match the provided keywords.

- `update_recomm(internship, CVID[ ])`

Updates recommendations for the provided internship with the given CVs (and vice versa) and returns information about the corresponding students.

- `get_internship(internshipID)`

Retrieves all information related to a published internship (recommended CVs, interested students, completed matches...).

- `remove_internship(internshipID)`

Deletes the internship corresponding to the provided identifier from the database.

- `remove_recomm_CV(internshipID)`

Removes the internship corresponding to the provided identifier from all CVs for which the first is recommended.

- `select_CV(internshipID, CVID)`

Adds the specified CV to the "selected CVs" for a given internship and retrieves information about the corresponding student and internship.

- `get_recom(CVID)`

Finds all recommended internships for the student associated with the provided CV identifier.

- `select_int(internshipID, CVID)`

Adds the specified internship to the "selected internships" for a given student (i.e. the one associated with the provided CV identifier) and retrieves information about the corresponding company and CV.

- `interested_companies(CVID)`

Finds all internships for which companies have selected the given CV.

- `accept_int(internshipID, CVID)`

Creates a match between the provided CV and internship (i.e. the ones associated to the specified identifiers). Retrieves information about the corresponding student and company and about the match creation.

- `save_quest(quests)`

Saves the provided questionnaires in the database.

- `accept_CV(internshipID, CVID)`

Creates a match between the provided internship and CV (i.e. the ones associated to the specified identifiers). Retrieves information about the corresponding company and student and about the match creation.

- `check_date(matchID, interview)`

Finds an interview (among the ones set up by the company involved in the provided match) whose date and time match those specified in the *interview* parameter.

- `setup_interview(matchID, interview, quest)`

Creates a new interview using information specified in the *interview* parameter and associates it with the provided match and questionnaire. Saves the provided questionnaire in the database. Retrieves information about the interviewed student.

- `get_calendar(studentID)`

Retrieves all information regarding the interviews in which the specified student (i.e. the one associated with the provided identifier) is involved.

- `update_rank(questionnaire)`

Updates the provided selection process questionnaire in the database with the assigned grade and retrieves information about the involved student and company.

- `admit_student(matchID)`

Activates an internship program based on the provided match and retrieves information about the activated internship and the involved student and company.

- `reject_student(matchID)`

Rejects a student for an internship program based on the provided match and retrieves information about the match and the involved student.

- `get_sat_requests(studentID)`

Retrieves all satisfaction questionnaires regarding matches and active internships in which the specified student is involved.

- `save_sat_quest(questionnaire)`

Updates the provided satisfaction questionnaire in the database with the assigned evaluation.

- `update_recomm(internshipID[ ],CVID[ ])`

Updates recommendations for the provided CVs with the given internships (and vice versa) and retrieves information about the corresponding companies, students and internships.

- `get_stud_messages(studentID)`

Retrieves all messaging sessions related to the specified student.

- `save_stud_mex(mex, studentID, internshipID)`

Using the provided student message, it updates the messaging session related to the active internship involving the specified student and internship. It also retrieves information about the corresponding student, company and internship.

- `get_int_messages(internshipID)`

Retrieves all messaging sessions related to the specified internship.

- `save_comp_mex(mex, studentID, internshipID)`

Using the provided company message, it updates the messaging session related to the active internship involving the specified student and internship. It also retrieves information about the corresponding student and internship.

2.6. Selected Architectural Styles and Patterns

In this section, the main architectural styles and patterns selected for the S&C application and the principal reasons that led to these choices are described.

4-tier Architecture

As mentioned in Section 2.1, the high level architectural style designed for the application is a 4-tier architecture, which enriches the well known 3-tier architecture inserting an intermediate layer (Web Server) between the Application Server and the Presentation Layer. First of all, the presence of a Web Server, whose main functionalities are to manage and route HTTPS request to the Application Server endpoints and serve static web content, increases the logical separation of the layers, simplifying development, debugging and future maintenance. This architectural choice has also a big advantage in terms of security, which can be improved by adding a supplementary level of firewalls between the Presentation and the Business Logic Layer (details in Figure 2.3), protecting the application server from potential external denial-to-service attacks. Web servers also exploit a caching mechanism to store static and frequently requested content (such as stylesheets or JavaScript files), in order to quicken delivery, reduce the load and optimize performance. In addition, all the advantages that characterize the 3-tier architectural style, such as separation of concerns, scalability and maintainability, are still preserved with a 4-tier Architecture.

REST API

RESTful APIs will be used for the interaction between the first three levels (Web Browser, Web Server and Application Server) of the S&C architecture. REST (Representational State Transfer) is a standardized and stateless architectural style built on the traditional HTTP protocol, which makes developers' work simple and intuitive, as they don't need to learn new communication protocols. As mentioned, REST is stateless, which means that requests sent from clients to servers must contain all the necessary information; this aspect enables servers to scale independently without needing to manage state sessions. Also caching mechanisms are supported, which, combined with REST statelessness, increases performance and reduces the average response time for clients' requests. These characteristics make the usage of RESTful APIs a powerful decision for managing the communication between the upper architectural layers of the S&C application.

Façade Pattern

As highlighted in the Component Diagram in Figure 2.2, a Façade Pattern has been used to reduce dependencies and enhance loose coupling between clients and the Application Server. Except for simple operations such as CV upload/removal or internship publication/removal, the Web Server, after receiving requests from web browsers, does not directly interact with low level components (Recommendation Manager, Matching Manager, Notification Manager, Messaging Manager and Selection Manager) but communicates with two high level façades, represented by the Company Manager and the Student Manager. These latter "hide" the low level ones, reducing the number of dependencies between external and application components, also facilitating code changes and maintenance, as developers can modify the internal subsystem without the need of changing exposed interfaces.

2.7. Other Design Decisions

This section presents other design decisions, exploited to improve non-functional requirements, such as modularity, availability and performance.

MongoDB

For the Data Layer of the application's architecture, the chosen database technology is MongoDB, a distributed, schema-less and scalable documental DBMS. First of all, the native document-oriented approach adopted by this technology enables the storage of complex business objects as JSON files, which represent a suitable and simple representation for data entities such as student CVs or internship documents used in the S&C application. MongoDB is designed with a layered architecture, based on a first level of servers (mongos) which collect user query requests and route them to the right data node of a distributed cluster of shards. In a single shard, a local DBMS (mongod) computes and executes the queries and returns the output back to the mongos servers, which then show the content to the users or send it back to the application server in the case of S&C. One of the advantages of this architecture is that mongos works as a balancer for the shards of the cluster, avoiding the necessity of an external balancing mechanism (as described in the Deployment Diagram in Figure 2.3) between the Application Server and the data nodes, therefore promoting resource optimization. The main reason that led to the usage of MongoDB is Atlas Search, an integrated capability that enables full-text and relevance-based search on textual data directly through MongoDB queries. This feature considerably simplifies the implementation of the recommendation manager, which doesn't need an

internal search engine to execute keyword-matching between internship documents and student CVs, but can obtain the same result by performing simple database queries.

Replication

A replication strategy has been applied to all major entities of the application deployment (Web Server, Application Server, load balancers and firewalls). This choice enhances fault tolerance and availability, ensuring that logical components remain always operational even if some instances fail. Replication also represents an advantage in terms of scalability, as the system can easily handle increasing loads of requests by distributing them across the different replicas.

3 | User Interface Design

This section provides the mockups of the main user interfaces, to be used as a guide during the implementation phase.

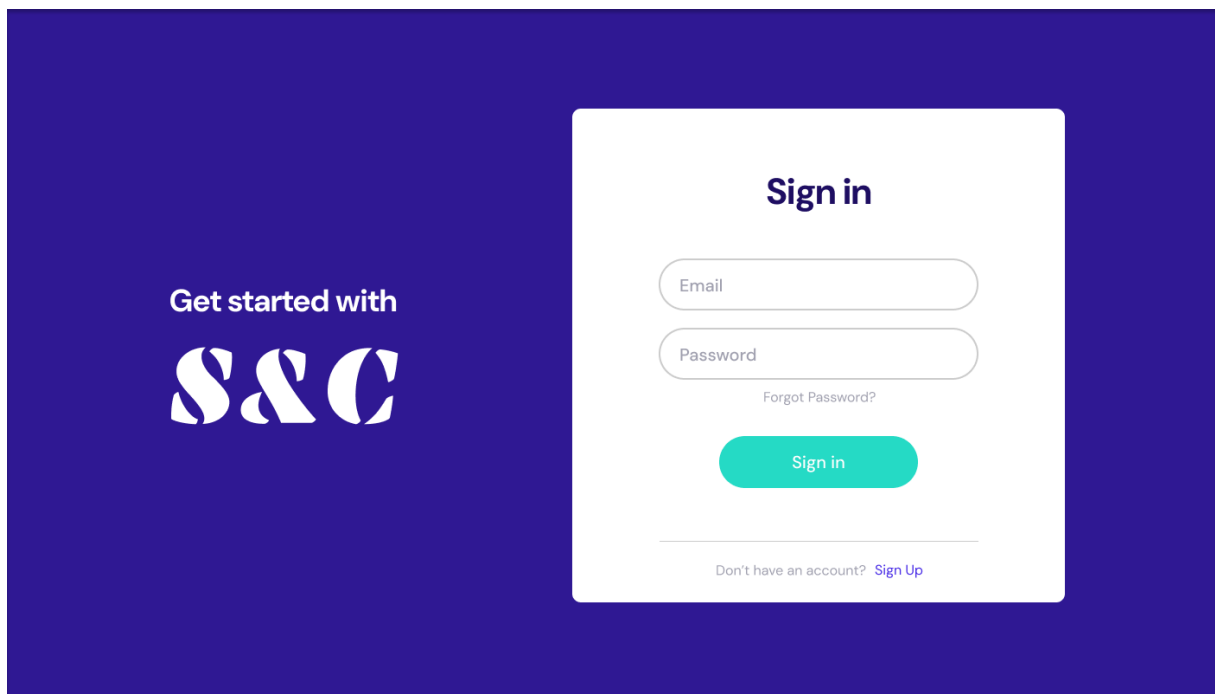
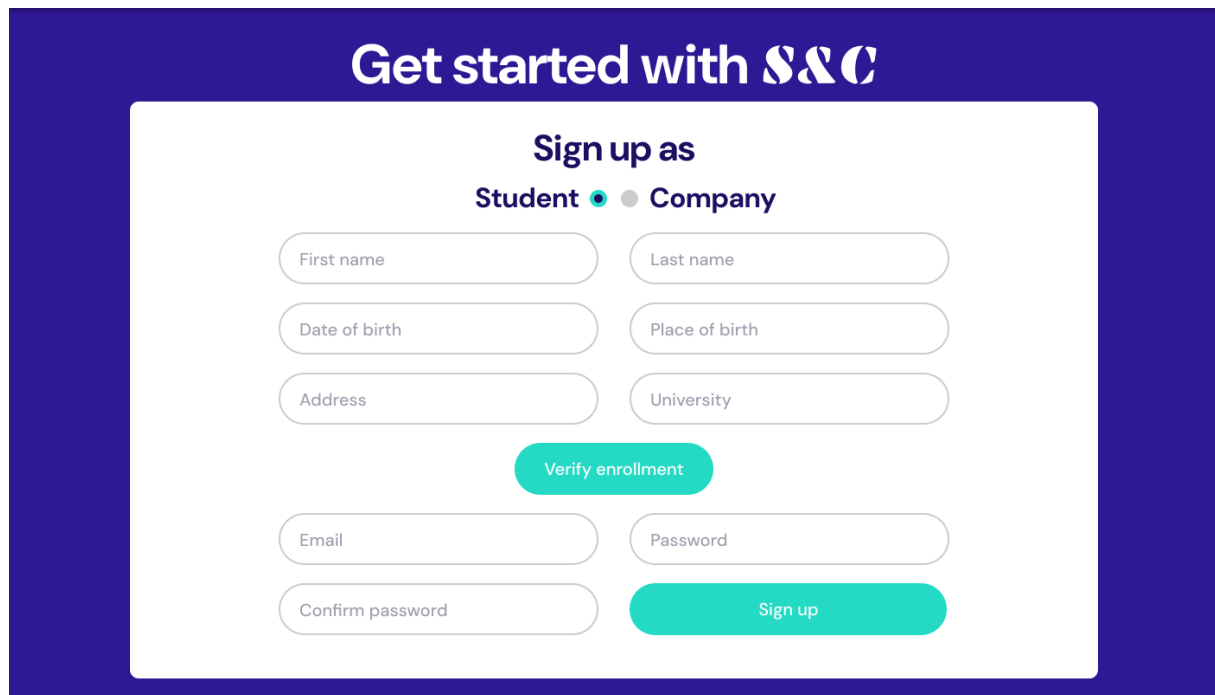


Figure 3.1: Login page UI

The login page, common to both students and companies, is shown in Figure 3.1.

Figures 3.2 and 3.3 show the registration interfaces of students and companies. By clicking on "Verify enrollment", students are redirected to an external platform where they can authenticate using their institutional credentials.



Get started with S&C

Sign up as
Student ● ● **Company**

First name Last name

Date of birth Place of birth

Address University

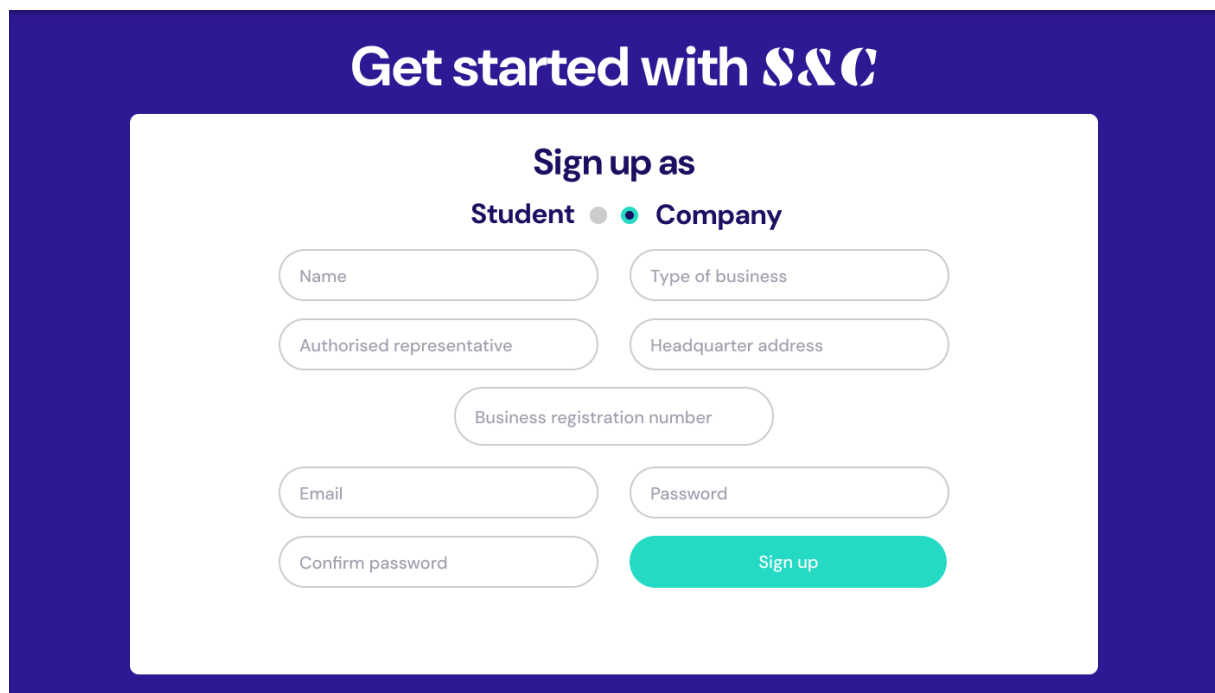
Verify enrollment

Email Password

Confirm password Sign up

The image shows a registration form for 'S&C' with a dark blue background. The form is titled 'Get started with S&C' and 'Sign up as'. It has two radio buttons for 'Student' (selected) and 'Company'. The form fields are arranged in two columns: First name, Last name, Date of birth, Place of birth, Address, and University. There is a 'Verify enrollment' button in the center. Below the columns are Email, Password, and Confirm password fields, followed by a 'Sign up' button.

Figure 3.2: Student registration UI



Get started with S&C

Sign up as
Student ● ● **Company**

Name Type of business

Authorised representative Headquarter address

Business registration number

Email Password

Confirm password Sign up

The image shows a registration form for 'S&C' with a dark blue background. The form is titled 'Get started with S&C' and 'Sign up as'. It has two radio buttons for 'Student' and 'Company' (selected). The form fields are arranged in two columns: Name, Type of business, Authorised representative, and Headquarter address. There is a 'Business registration number' field in the center. Below the columns are Email, Password, and Confirm password fields, followed by a 'Sign up' button.

Figure 3.3: Company registration UI

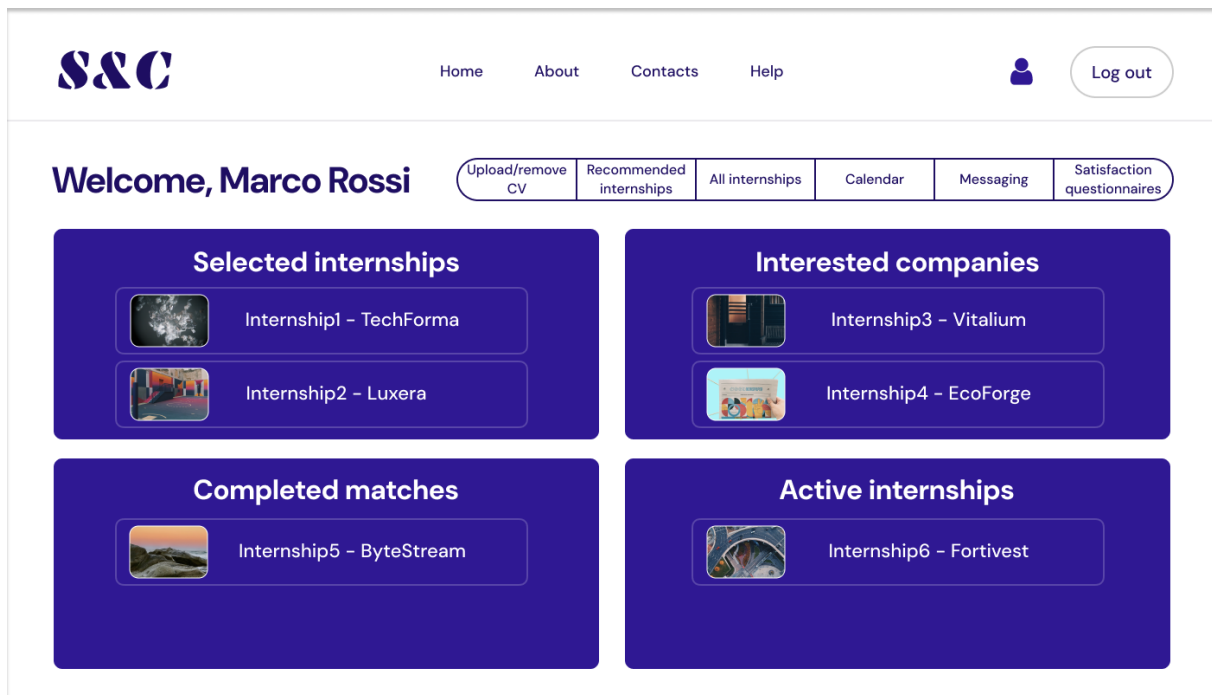


Figure 3.4: Student homepage UI

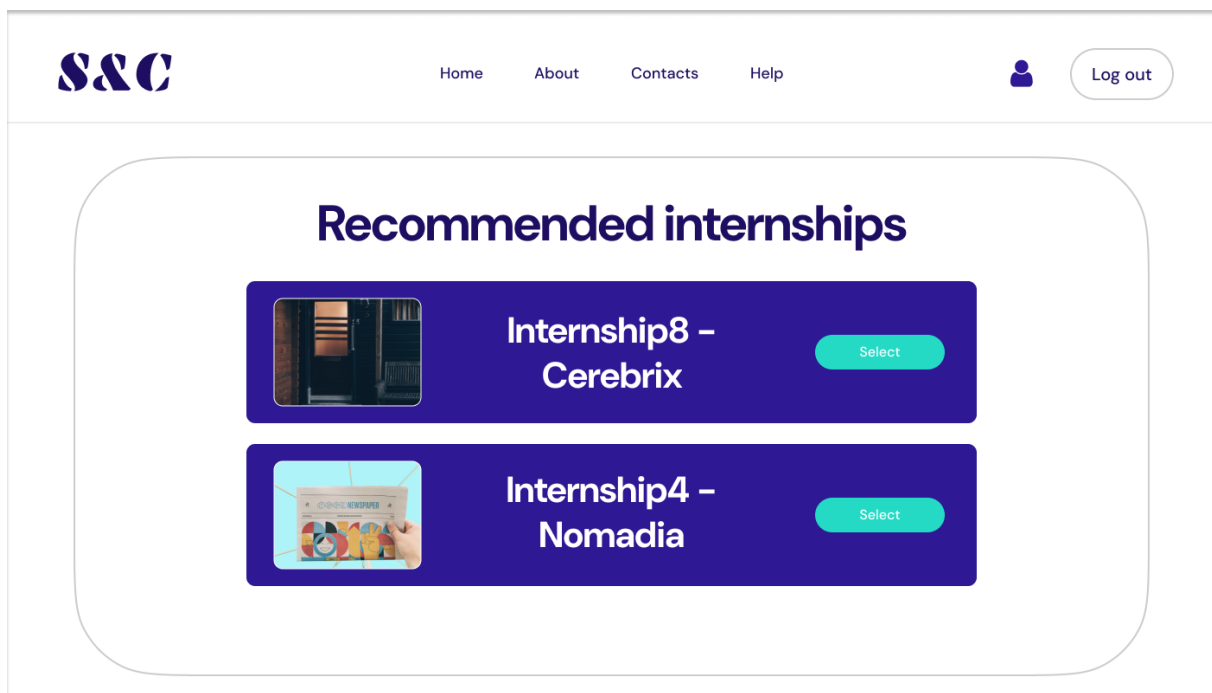


Figure 3.5: Student recommended internships UI

In Figure 3.4 students homepage interface is shown. It provides an overview of the main sections they can access.

Figure 3.5 shows the recommended internships section for students. Note that, even if not reported in this document, a totally analogous interface for companies can be considered with recommended students for each of their internships.

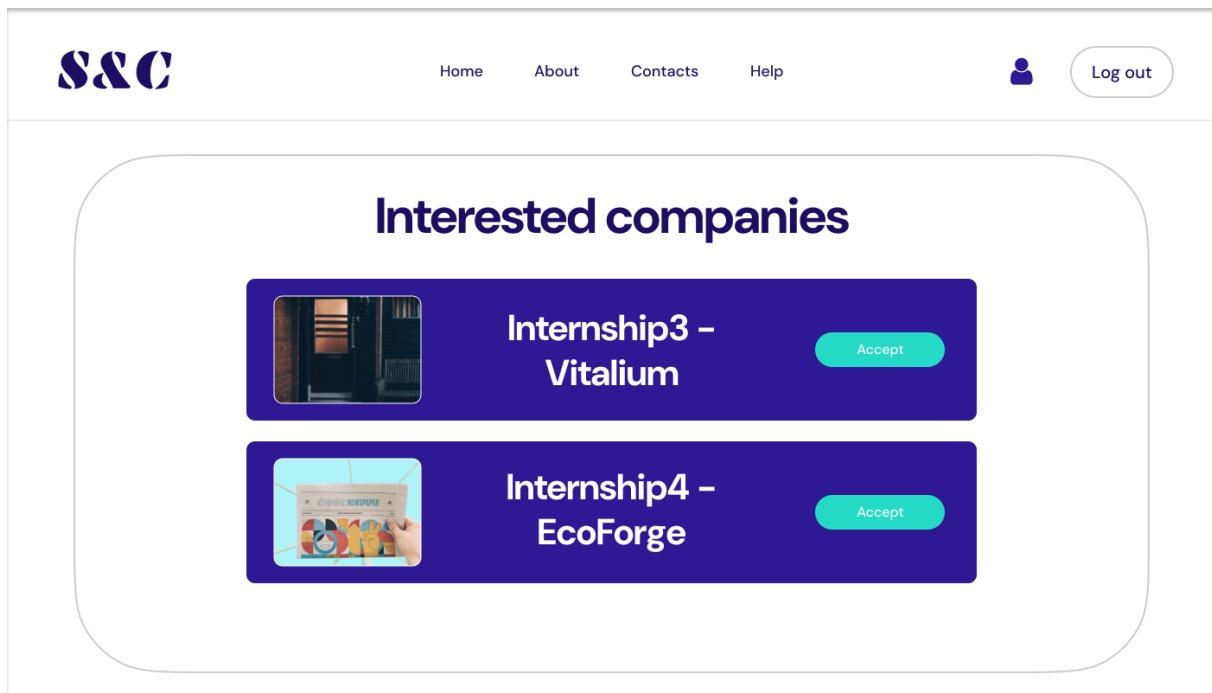


Figure 3.6: Student interested companies UI

Figure 3.6 shows the interested companies section for students. Note that, even if not reported in this document, a totally analogous interface for companies can be considered with interested students for each of their internships.

In Figure 3.7 the student interview calendar is shown, just after clicking on a specific interview. Note that, even if not reported in this document, a totally analogous interface can be considered for companies, simply without the "Fill out questionnaire" option.

Figure 3.8 shows the student messaging section, containing a chat for each of their active internships. Note that, even if not reported in this document, a totally analogous interface for each internship can be considered for companies, with a chat for every enrolled student.

Figure 3.9 shows the student interface for the satisfaction questionnaires section. Note that, even if not reported in this document, a totally analogous interface for each internship can be considered for companies.

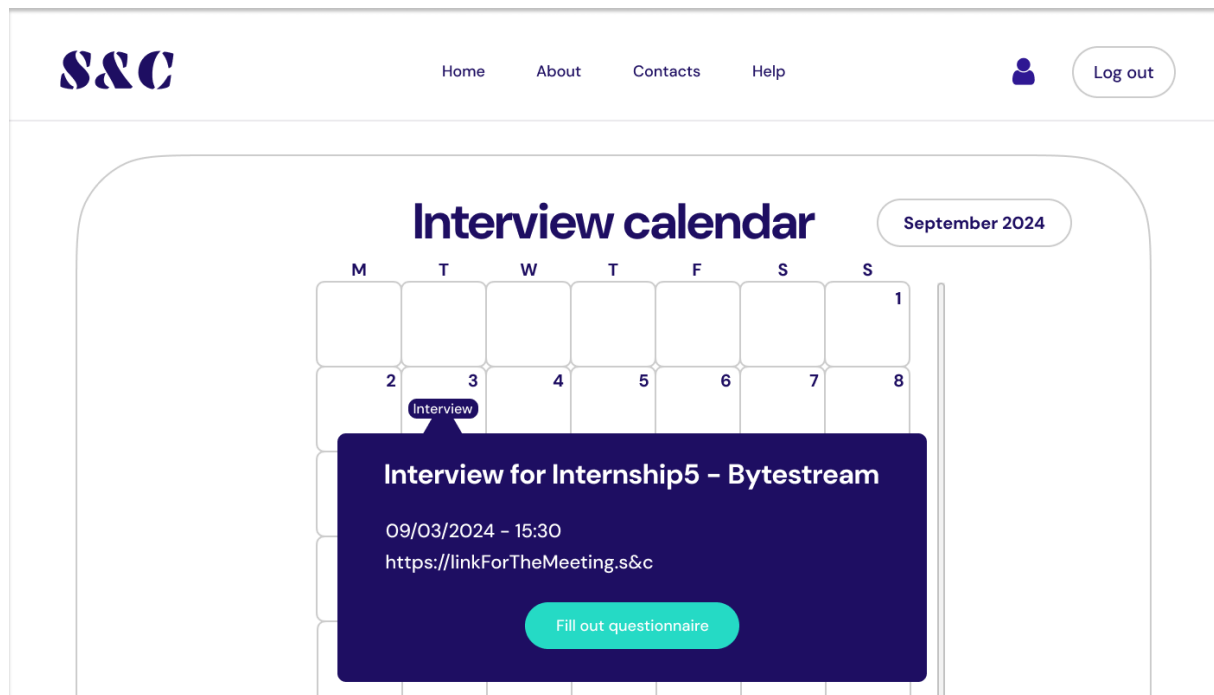


Figure 3.7: Student interview calendar UI

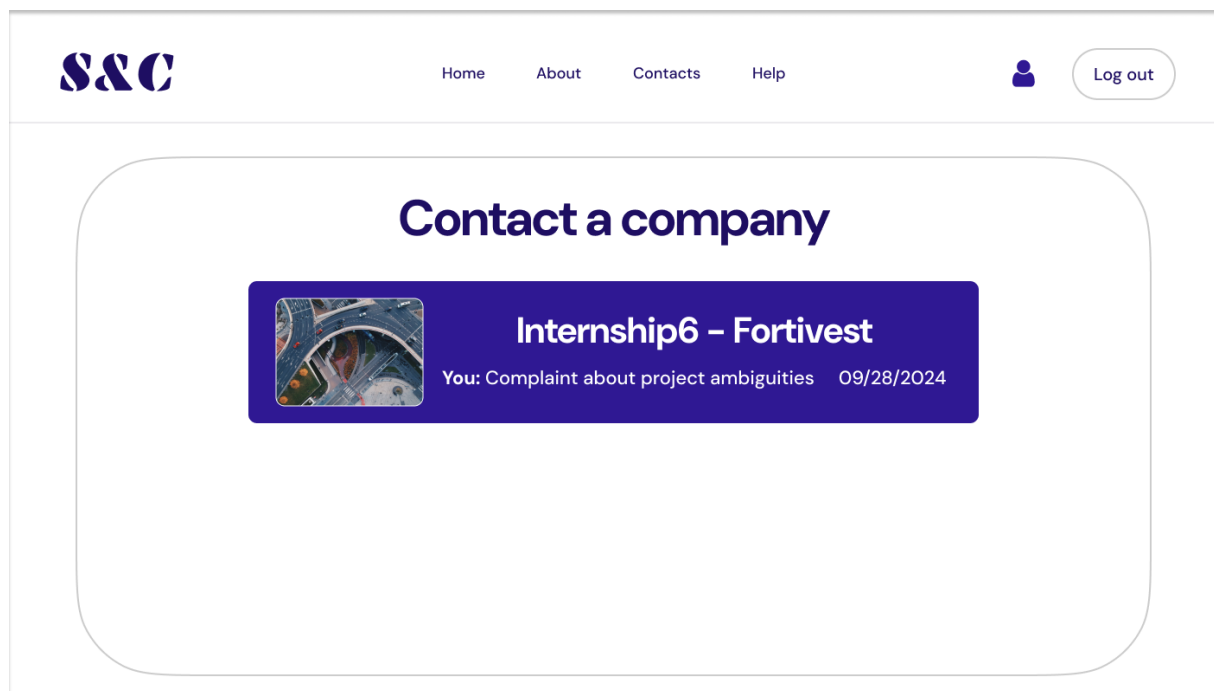


Figure 3.8: Student messaging section UI

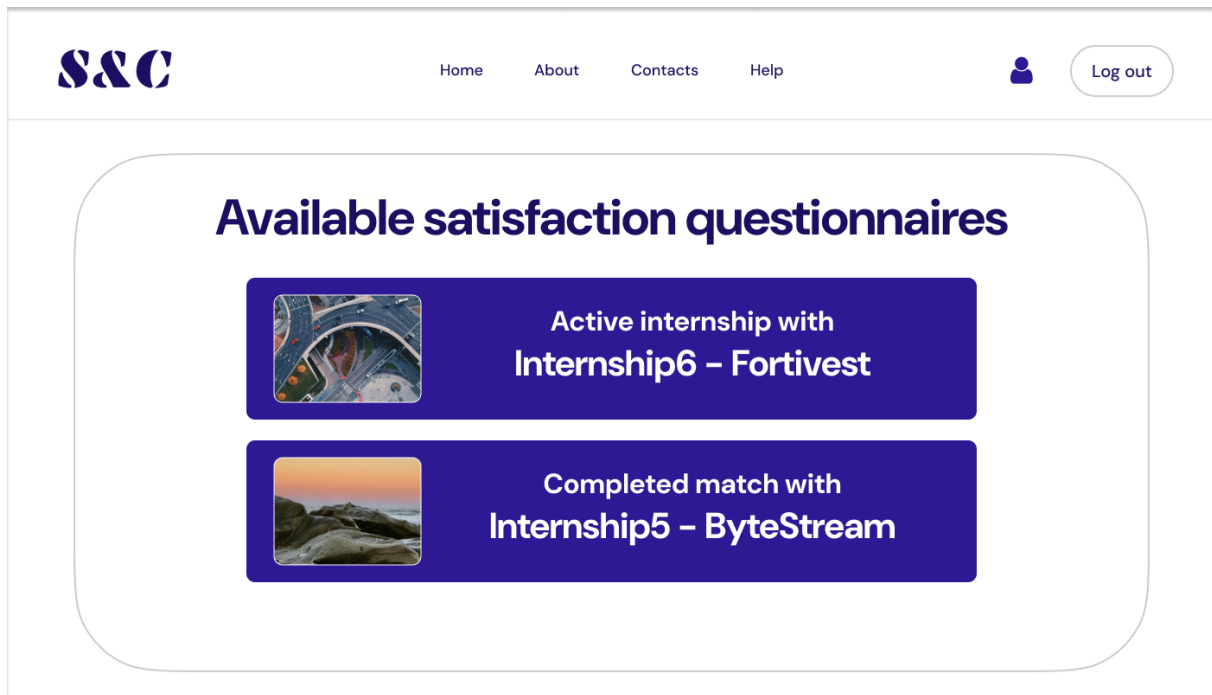


Figure 3.9: Student satisfaction questionnaires section UI

In Figure 3.10 the company homepage interface is shown. From there, companies can access a section dedicated to each of their internships.

Figure 3.11 shows the internship section interface for companies. It provides an overview of the main sections they can access.

Figure 3.12 shows the company interface dedicated to completed matches for an internship. From there, companies can set up interviews and admit/reject students based on the suitability rank.

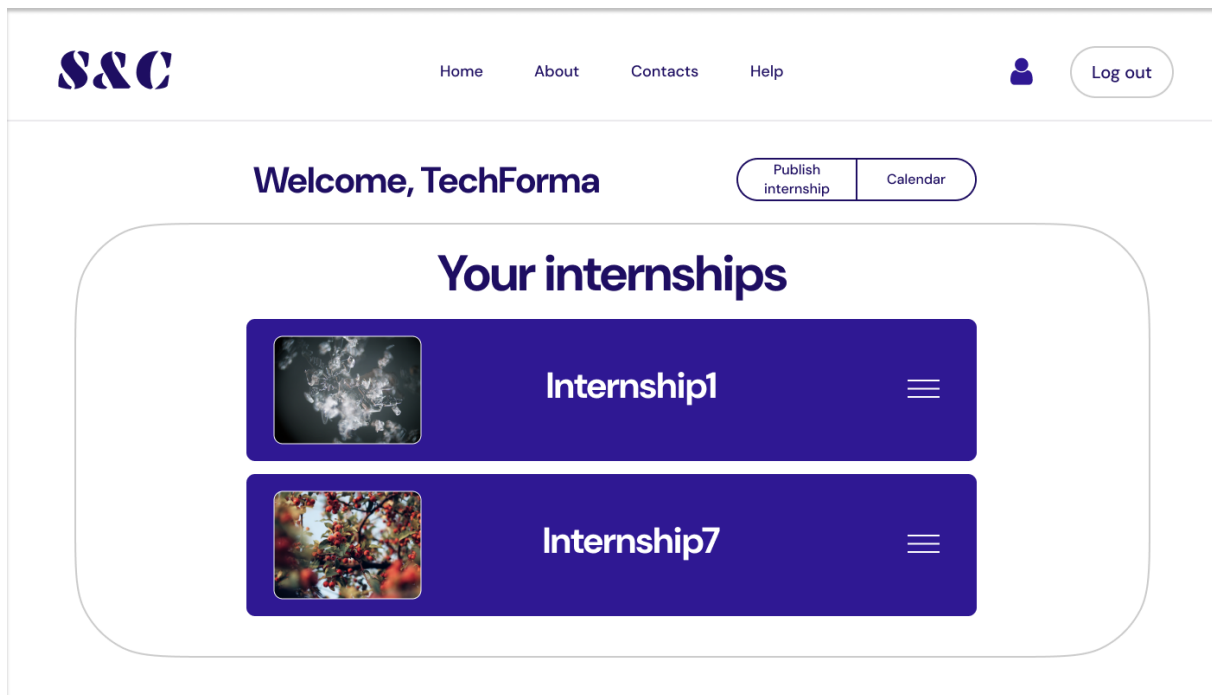


Figure 3.10: Company homepage UI

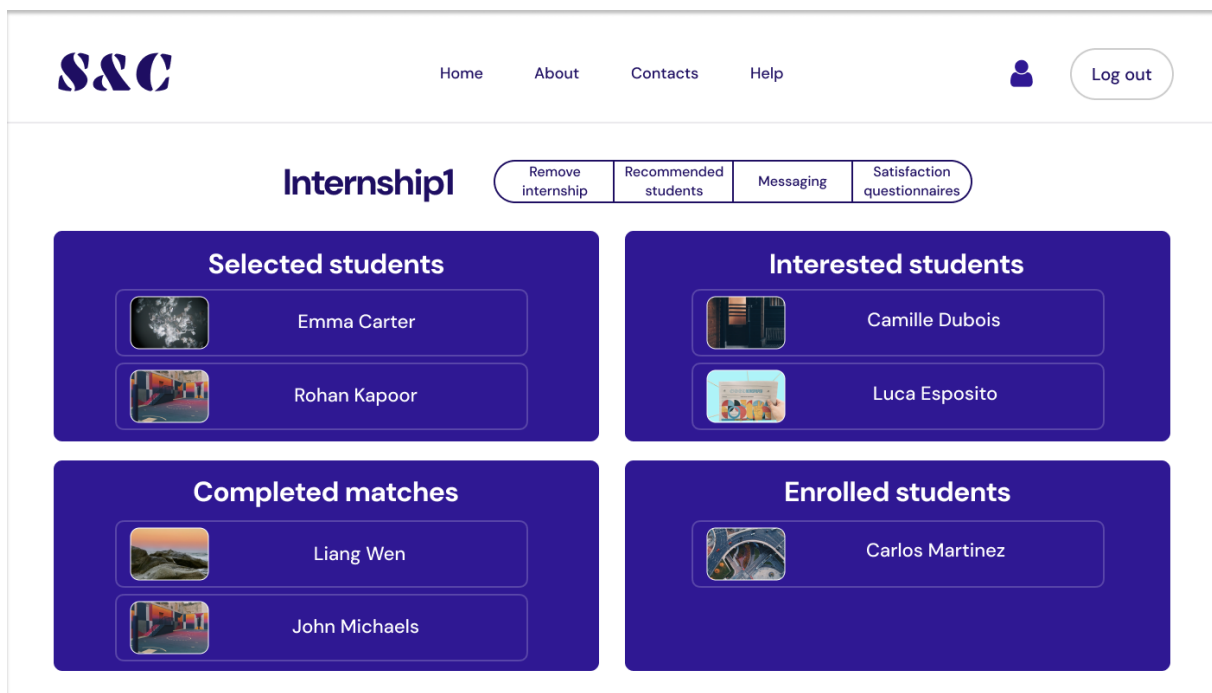


Figure 3.11: Company internship section UI

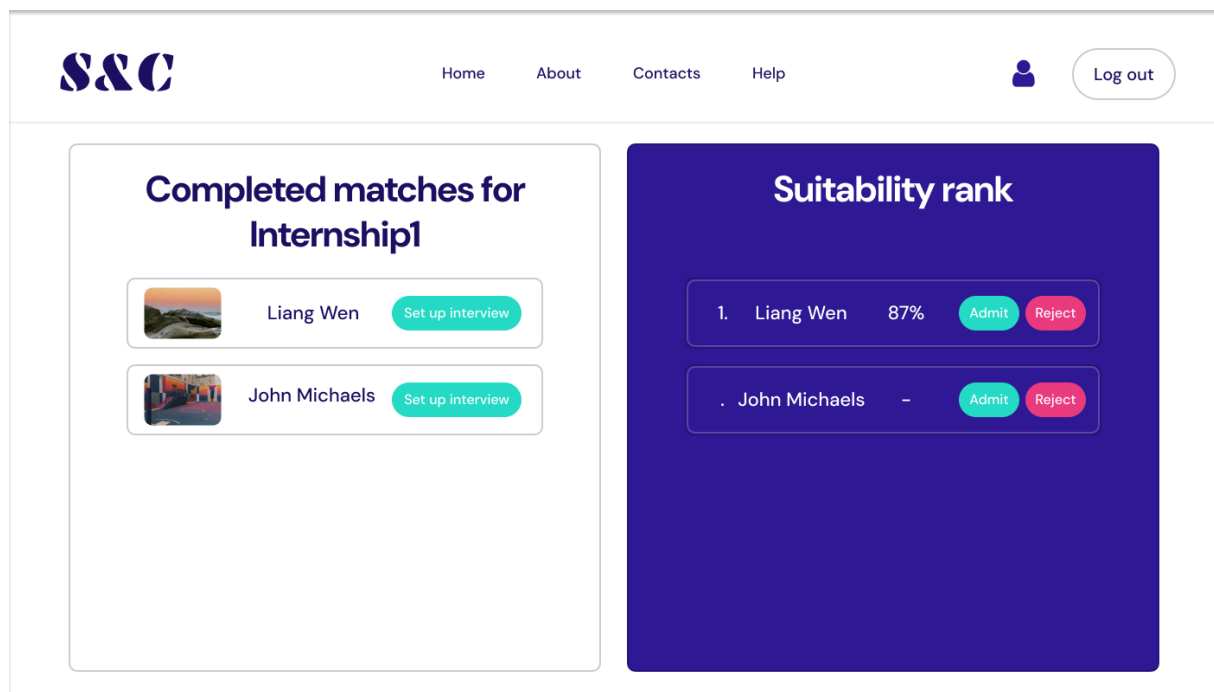


Figure 3.12: Company completed matches UI

4 | Requirements Traceability

In this section, it is shown how the functional requirements defined in the RASD of the S&C application map the design components described in Section 2.2.

Sign up, log in, log out

[R1] The system should allow new users (i.e. they are not registered yet) to register as students by providing their personal information, a valid email address and a password¹.

Components: WebBrowser, WebServer, AuthenticationManager, DataAccessManager, StudentVerificationPlatform, DBMS.

[R2] The system should allow new users (i.e. they are not registered yet) to register as companies by providing company identification information, a valid email address and a password².

Components: WebBrowser, WebServer, AuthenticationManager, DataAccessManager, CompanyVerificationPlatform, DBMS.

[R3] The system should allow registered users to log in using their email address and password.

Components: WebBrowser, WebServer, AuthenticationManager, DataAccessManager, DBMS.

[R4] The system should allow all logged in users to log out from the platform.

Components: WebBrowser, WebServer, AuthenticationManager, DataAccessManager, DBMS.

¹See authentication requirement R54.

²See authentication requirement R55.

Students

[R5] The system should allow students to upload their CV on the platform only if a CV of that student is not currently uploaded.

Components: WebBrowser, WebServer, StudentManager, DataAccessManager, DBMS.

[R6] The system should allow students to remove their uploaded CV from the platform³.

Components: WebBrowser, WebServer, StudentManager, DataAccessManager, DBMS.

[R7] The system should allow students to look among all available internships (and related document) published by companies on the platform.

Components: WebBrowser, WebServer, StudentManager, DataAccessManager, DBMS.

[R8] The system should allow students to look among internships (and related document) that are recommended by the platform.

Components: WebBrowser, WebServer, StudentManager, DataAccessManager, DBMS.

[R9] The system should allow students who have already uploaded their CV to select one or more internships of their interest among the available ones (i.e. students start the matching process showing interest for the internship).

Components: WebBrowser, WebServer, StudentManager, MatchingManager, DataAccessManager, DBMS.

[R10] The system should notify students whenever a new recommended internship has been published on the platform by a company.

Components: RecommendationManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

[R11] The system should notify students whenever a company, whose internship has been selected by that student (i.e. the student initiated the process), has accepted their CV for the selection process regarding that internship.

Components: MatchingManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

³The modification of a CV can be obtained by combining R5 and R6.

[R12] The system should notify students whenever a company selects their CV for an internship (i.e. the company initiates the process showing interest for a student CV).

Components: CompanyManager, NotificationManager, MatchingManager, DataAccessManager, DBMS.

[R13] The system should allow students to accept an internship whose company selected their CV for that internship (i.e. students end the matching process, the selection process starts).

Components: WebBrowser, WebServer, StudentManager, MatchingManager, DataAccessManager, DBMS.

[R14] The system should notify students whenever an interview for one of their matching internships has been programmed by a company.

Components: SelectionManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

[R15] The system should allow students to see their programmed interviews on a calendar.

Components: WebBrowser, WebServer, StudentManager, SelectionManager, DataAccessManager, DBMS.

[R16] The system should notify students whenever a questionnaire for one of their matching internships has been uploaded by a company.

Components: SelectionManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

[R17] The system should allow students to fill out available questionnaires for their matching internships.

Components: WebBrowser, WebServer, StudentManager, SelectionManager, DataAccessManager, DBMS.

[R18] The system should notify students when they are admitted to a matching internship after their interview with the company.

Components: SelectionManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

[R19] The system should notify students when they are rejected for a matching internship after their interview with the company.

Components: SelectionManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

[R20] The system should allow students to provide information to the other party about their active internships (such as complaints, problems or feedback).

Components: WebBrowser, WebServer, StudentManager, MessagingManager, DataAccessManager, DBMS.

[R21] The system should notify students whenever a company provides information (such as complaints, problems or feedback) about an active internship in which they are involved.

Components: MessagingManager, CompanyManager, NotificationManager, DataAccessManager, DBMS.

[R22] The system should notify students whenever a satisfaction questionnaire regarding an active internship or a matching process is available.

Components: RecommendationManager, NotificationManager, StudentManager, CompanyManager, DataAccessManager, DBMS.

[R23] The system should allow students to fill out available satisfaction questionnaires.

Components: WebBrowser, WebServer, StudentManager, RecommendationManager, DataAccessManager, DBMS.

Companies

[R24] The system should allow companies to publish their internships on the platform by providing a document containing information about projects and terms of the internship.

Components: WebBrowser, WebServer, CompanyManager, DataAccessManager, DBMS.

[R25] The system should allow companies to remove previously published internships from the platform⁴.

⁴The modification of an internship can be obtained by combining R24 and R25.

Components: WebBrowser, WebServer, CompanyManager, DataAccessManager, DBMS.

[R26] For each of their internships, the system should allow companies to look among available students' CVs recommended by the platform.

Components: WebBrowser, WebServer, CompanyManager, DataAccessManager, DBMS.

[R27] For each of their internships, the system should allow companies to select one or more CVs among the recommended ones (i.e. the company initiates the matching process showing interest for a CV).

Components: WebBrowser, WebServer, CompanyManager, MatchingManager, DataAccessManager, DBMS.

[R28] The system should notify companies whenever a new recommended CV for one of their internships has been uploaded by a student.

Components: RecommendationManager, StudentManager, NotificationManager, DataAccessManager, DBMS.

[R29] The system should notify companies whenever a student, whose CV has been selected by that company for a specific internship (i.e. the company initiated the matching process), has accepted that internship (i.e. the selection process starts).

Components: MatchingManager, StudentManager, NotificationManager, DataAccessManager, DBMS.

[R30] The system should notify companies whenever a student selects one of their internships (i.e. the student initiates the matching process showing interest).

Components: MatchingManager, StudentManager, NotificationManager, DataAccessManager, DBMS.

[R31] The system should allow companies to accept a CV of a student that selected one of their internships (i.e. companies end the matching process, starting the selection process).

Components: WebBrowser, WebServer, CompanyManager, MatchingManager, DataAccessManager, DBMS.

[R32] For each of their internships, the system should allow companies to set up an interview with a matching student (for that internship), providing date, time and a web link for the meeting.

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R33] The system should allow companies to see their programmed interviews on a calendar.

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R34] For each of their internships, the system should allow companies to create structured questionnaires for the selection process of matching students.

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R35] The system should allow companies to upload created questionnaires, making them available for the matching students.

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R36] The system should notify companies whenever a matching student filled out the questionnaire for the selection process regarding one of their internships.

Components: SelectionManager, StudentManager, NotificationManager, DataAccessManager, DBMS.

[R37] For each of their internships, the system should allow companies to see the results of the selection process questionnaires filled out by matching students ordered by grade.

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R38] For each of their internships, the system should allow companies to admit one or more interviewed matching students (i.e. the internship is now active).

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R39] For each of their internships, the system should allow companies to reject one or more interviewed matching students.

Components: WebBrowser, WebServer, CompanyManager, SelectionManager, DataAccessManager, DBMS.

[R40] For each student carrying out an internship with them, the system should allow companies to provide information about that internship (such as complaints, problems or feedback).

Components: WebBrowser, WebServer, CompanyManager, MessagingManager, DataAccessManager, DBMS.

[R41] For each of their active internships, the system should notify companies whenever a student involved in it provides information (such as complaints, problems or feedback).

Components: MessagingManager, StudentManager, NotificationManager, DataAccessManager, DBMS.

[R42] The system should notify companies whenever a satisfaction questionnaire regarding an internship or a matching process is available.

Components: RecommendationManager, NotificationManager, StudentManager, CompanyManager, DataAccessManager, DBMS.

[R43] The system should allow companies to fill out available satisfaction questionnaires.

Components: WebBrowser, WebServer, CompanyManager, RecommendationManager, DataAccessManager, DBMS.

System

[R44] The system should be able to evaluate with a grade selection process questionnaires filled out by students.

Components: SelectionManager, DataAccessManager, DBMS.

[R45] As soon as an internship is activated, the system should provide the involved student and company with a satisfaction questionnaire in order to collect statistical information about that internship (such as feedback and suggestions).

Components: RecommendationManager, DataAccessManager, DBMS.

[R46] As soon as a student and company match, the system should provide both with a satisfaction questionnaire in order to collect information about the related matching process (such as feedback and suggestions).

Components: RecommendationManager, DataAccessManager, DBMS.

[R47] The system should be able to recommend published internships to the students by matching keywords between their CVs and the internships' documents.

Components: RecommendationManager, DataAccessManager, DBMS.

[R48] For each internship, the system should be able to recommend to the corresponding company available students' CVs by matching keywords between those and the internship document.

Components: RecommendationManager, DataAccessManager, DBMS.

[R49] The system should be able to evaluate (positively or negatively) a completed match between a student and an internship offered by a company by analyzing the satisfaction questionnaires filled out by both parties.

Components: RecommendationManager, DataAccessManager, DBMS.

[R50] The system should be able to associate students' CVs to similar ones involved in past completed matches (with internships) by keyword matching among CVs.

Components: RecommendationManager, DataAccessManager, DBMS.

[R51] The system should be able to associate internships to similar ones involved in past completed matches (with students' CVs) by keyword matching among internships' documents.

Components: RecommendationManager, DataAccessManager, DBMS.

[R52] The system should be able to recommend to students internships similar (i.e. associated as in R51) to ones whose completed matching with similar (i.e. associated as in R50) students was positively evaluated.

Components: RecommendationManager, DataAccessManager, DBMS.

[R53] For each internship, the system should be able to recommend to the corresponding company students' CVs similar (i.e. associated as in R50) to ones whose completed matching with similar (i.e. associated as in R51) internships was positively evaluated.

Components: RecommendationManager, DataAccessManager, DBMS.

[R54] At the time of registration, the system should be able to validate student university enrollment by redirecting them to an external platform through an API, where they can authenticate using their institutional credentials.

Components: WebBrowser, WebServer, AuthenticationManager, StudentVerification-Platform.

[R55] At the time of registration, the system should be able to validate provided company information (such as registration number) through an external API.

Components: AuthenticationManager, CompanyVerificationPlatform.

5 | Implementation, Integration and Test Plan

This chapter provides a plan for the component implementation, along with their integration and testing. Note that build and test plan are tightly coupled, and they will therefore proceed hand in hand.

5.1. Plan

The implementation and testing plan for S&C will follow a combined **thread** and **bottom-up** strategy. Functional threads corresponding to main system features are identified and will be implemented and tested incrementally one after the other. The identification of functional threads will enable developers to provide users with intermediate deliverables for validation purposes, each one with new available functionalities, maximizing the visible progress. Within each thread, a bottom-up strategy will be carried out, thus starting from low level components and using drivers to simulate the behavior of higher level ones. This choice allows to progressively build each thread, unit testing each component and minimizing the delay in finding faults. During this process, only new feature core functionalities will be added, simplifying testing.

5.2. Features Identification

The main system features are outlined below, reflecting the product functions. Each of them corresponds to a single thread, that depends on the previous one. As already mentioned, their implementation and testing will proceed incrementally.

[F1] Sign up, log in, log out

The first feature includes the basic functionalities that allow users to register to the platform, log in and out. This will also include the integration with the external APIs providing student and company identity verification.

[F2] CV and internship management

The second feature includes the CV and internship management operations, namely their upload/publication and removal from the platform.

[F3] Recommendation

This third feature actually includes just a subset of the recommendation functionalities, in particular the one based on keyword matching across CVs and internships, since matches (and their evaluation) is still not available at this stage.

[F4] Matchmaking process

The fourth feature includes most of the matchmaking process functionalities, except for the ones that will be useful for the selection process. Specifically, CV/internship selection and match creation are covered here, together with new recommendation operations such as satisfaction questionnaires provision (and consequent evaluation).

[F5] Selection process

In the fifth feature, all core functionalities regarding the selection process for an internship are covered. These include interview set up, questionnaires management (creation, compilation, grading) and student admission/rejection (and consequent internship activation). Additionally, new satisfaction questionnaires for activated internships are provided in this phase.

[F6] Messaging

The sixth feature regards student-company communication, that happens after an internship has been activated. All functionalities regarding messages are included here.

[F7] Notifications

The seventh and final feature concerns notifications, that have been left for last since they do not impact the system main behavior. These cover all possible notifications throughout the entire flow of the application.

5.3. Component Integration and Testing

This section describes the flow to follow for the integration of components. As discussed before, within each thread, a bottom-up strategy will be adopted, starting from

components at a lower level in the diagrams, testing them individually and using drivers to simulate the ones above (for the sake of simplicity, only thread integration is shown). Moreover, note that users will be actively involved in the process, being provided with intermediate deliverables to be validated. To do so, also the development and the integration with components external to the application server (namely, DBMS, external API providers, web server and web browser) ought to be considered (frontend UI as well). In fact, diagrams below focus only on system components. As a reminder, the components offering REST APIs to the clients are the StudentManager, the CompanyManager and the AuthenticationManager.

To better frame the process, the whole diagram is always shown, with integrated threads added at each step. Also, note that colored portion sizes do not reflect the real percentage of features of the components.

[F1] Sign up, log in, log out

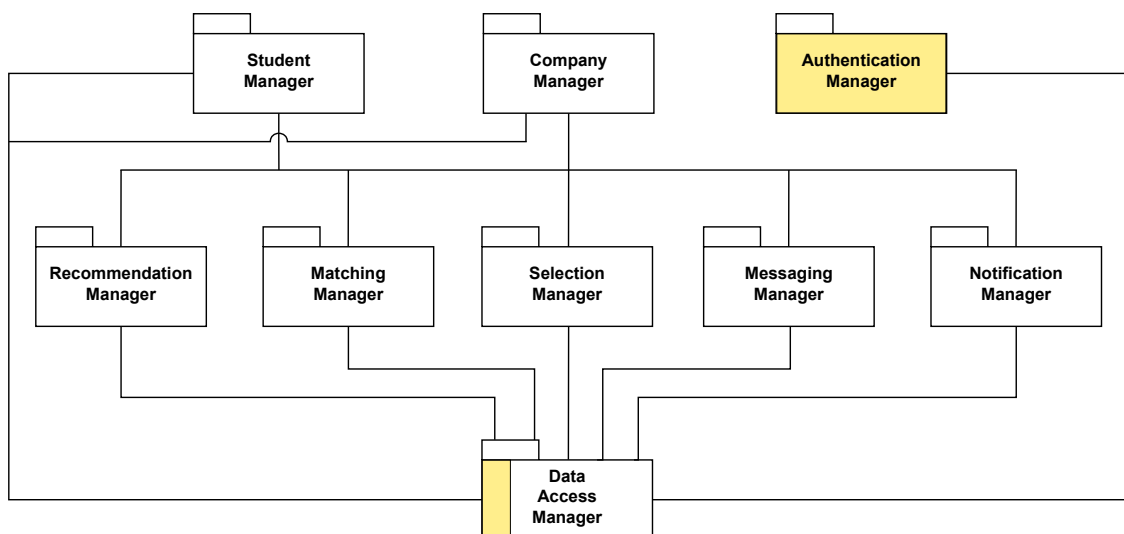


Figure 5.1: First thread integration

Figure 5.1 shows in yellow the integration of the first thread, regarding sign up, log in and log out functionalities. Required components to achieve it are a portion of the DataAccessManager (handling queries to store new user data, check credentials, etc.) and the AuthenticationManager. The latter will use APIs exposed by the external platforms aimed at verifying students' and companies' identities at the time of registration.

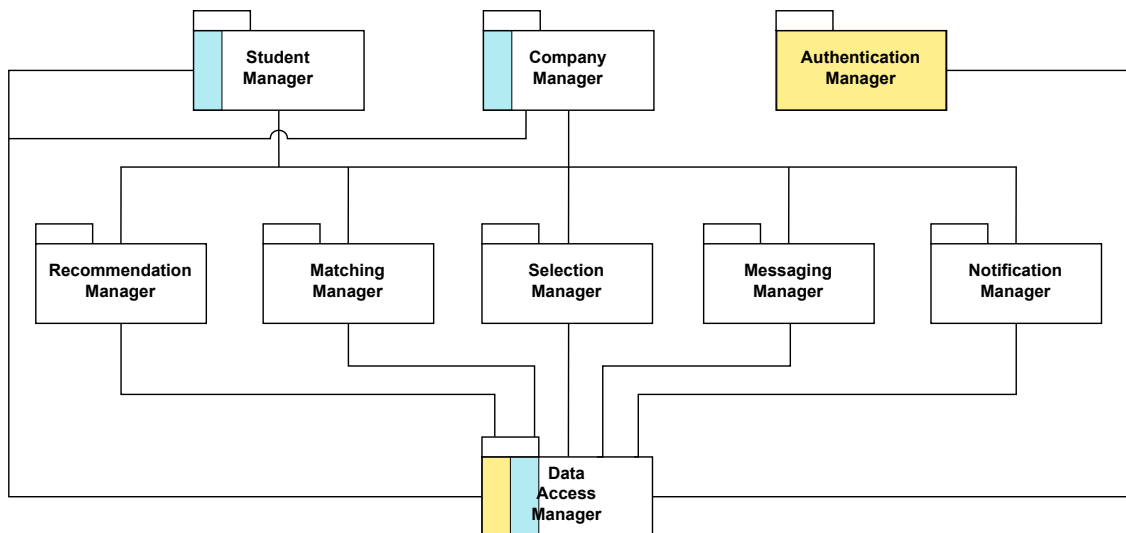
[F2] CV and internship management**Figure 5.2: Second thread integration**

Figure 5.2 shows in light blue the integration of the second thread. CV and internship management functionalities are handled by the StudentManager and the CompanyManager respectively. A new portion of the DataAccessManager for storing and retrieving new documents is also needed.

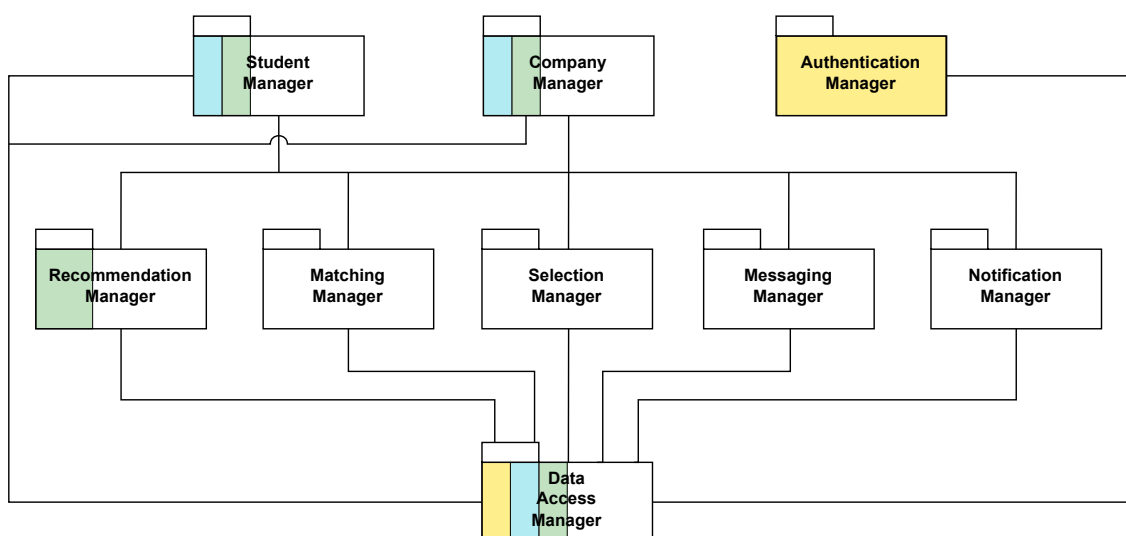
[F3] Recommendation**Figure 5.3: Third thread integration**

Figure 5.3 shows in green the integration of the third thread, that regards the first rec-

ommendation features based on keyword matching across CV and internship documents, handled by the RecommendationManager. It will be triggered either by the StudentManager or by the CompanyManager after new CVs/internships are uploaded on the platform, and will naturally need a new portion of the DataAccessManager.

[F4] Matchmaking process

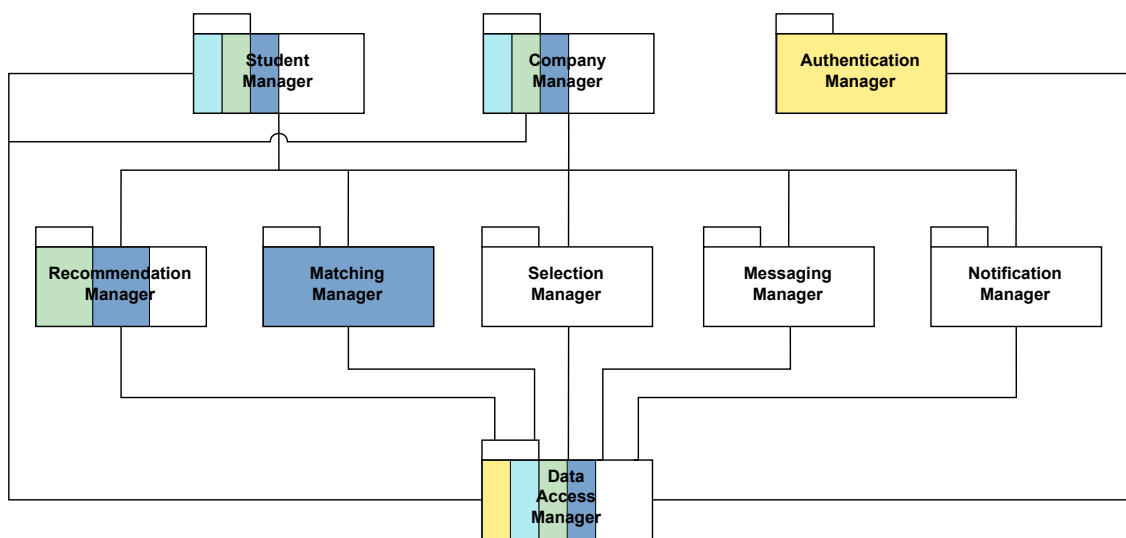


Figure 5.4: Fourth thread integration

Figure 5.4 shows in blue the integration of the fourth thread, concerning the matchmaking process. The MatchingManager, triggered by student and company managers, handles CV/internship selection, and eventually consequent match creation. Here, the RecommendationManager plays the role of handling satisfaction questionnaires for matches between students and internships (related companies), that can be filled out by both parties and be evaluated by the component to feed recommendation.

[F5] Selection process

Figure 5.5 shows in orange the integration of the fifth thread. It includes the whole selection process, which is handled by the namesake component. Also, the last RecommendationManager features are included here. They concern satisfaction questionnaires generated for activated internships, including their compilation, evaluation and consequent recommendations updates.

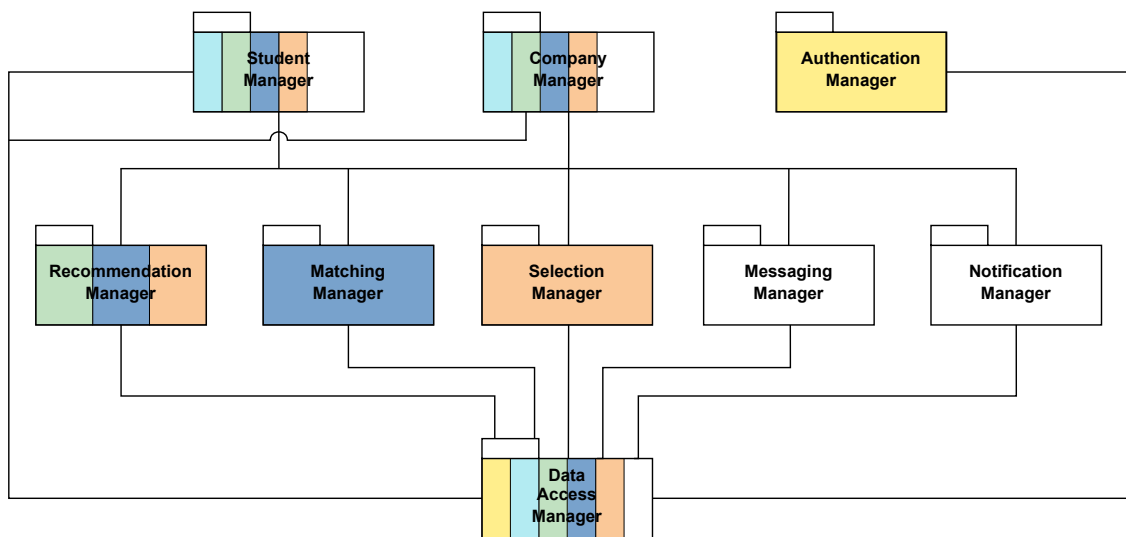


Figure 5.5: Fifth thread integration

[F6] Messaging

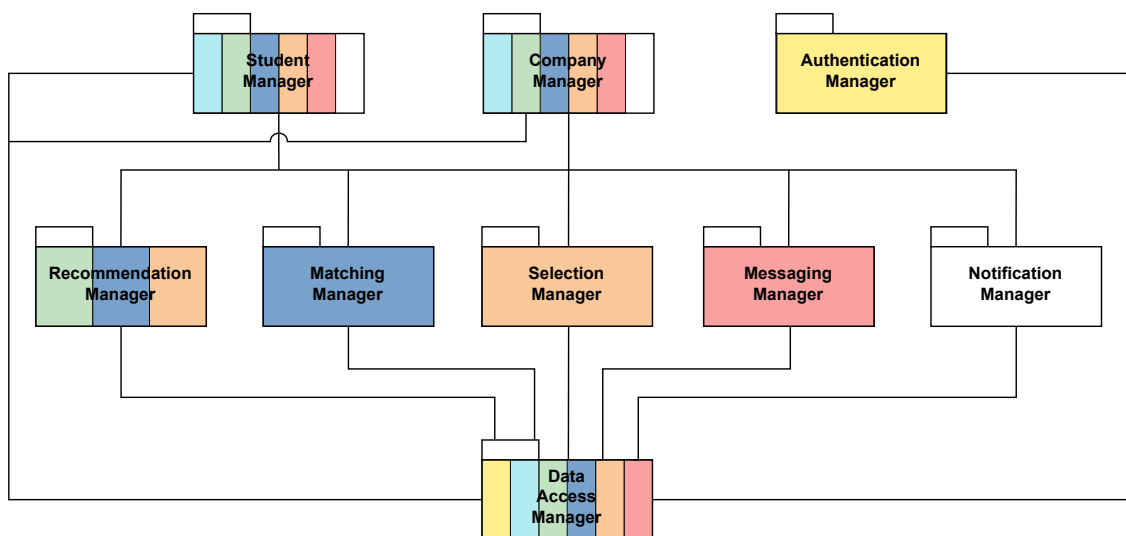


Figure 5.6: Sixth thread integration

Figure 5.6 shows in red the integration of the sixth thread, regarding student-company communication after internships are activated. This is handled by the MessagingManager, and, as for all other threads, new needed portions of StudentManager, CompanyManager and DataAccessManager are added.

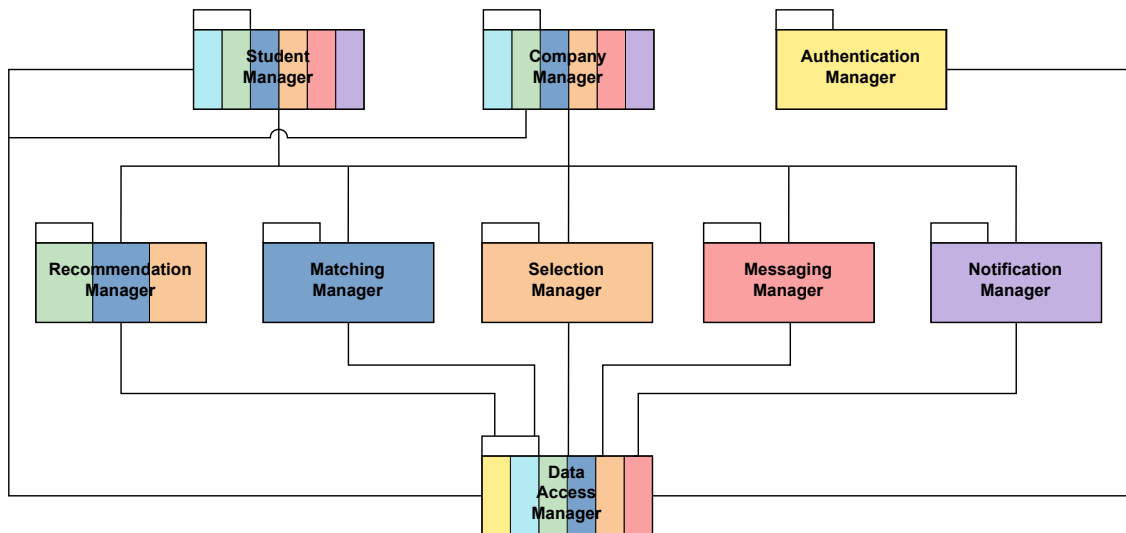
[F7] Notifications**Figure 5.7: Seventh thread integration**

Figure 5.7 shows in purple the integration of the seventh and final thread, which is totally focused on notifications. Student and company manager will now trigger the NotificationManager anytime users have to be notified, throughout the whole application flow.

5.4. System Testing

Once components have been unit tested and integrated with each other, system testing can be carried out. This process is aimed at verifying compliance with both functional and non-functional requirements that were specified in the RASD. The steps to be taken for the last testing part are outlined below.

Functional testing

Functional testing is intended to check whether functional requirements are fulfilled. To do so, all use cases described in the RASD will be executed and tested on the fully developed system. In addition, not only single requirements must be satisfied, but it should also be checked whether full logic flows of the application work as expected. For instance, the whole process leading to an internship activation could be simulated, starting from the CV (or internship) selection, passing through the match creation and ending with the selection process.

Performance testing

This phase of system testing is aimed at verifying performance requirements defined in the RASD, in particular the ones about response time and concurrent users handling. To achieve this, external tools¹ can be used to simulate real-world expected workloads and measure system performance. In addition to features providing an immediate user feedback, also background functionalities such as recommendation should be tested in this phase, to detect possible inefficiencies in the used algorithms.

Load testing

Rather than strictly focusing on performances, load testing aims at finding system's limits, also regarding single components. To accomplish this, the system will be tested continuously increasing the simulated workload and for long periods.

Stress testing

The last step of system testing is stress testing. The purpose here is to verify the ability of the system to recover. In order to induce failures, system testers will have to simulate high workloads of requests, exceeding the expected values that were indicated in the RASD, and possibly reduce resources, for instance decreasing the number of server replicas.

¹Among others, Apache JMeter is a useful software tool designed for performance and load testing.

6 | Effort Spent

This section presents an overview of the effort (calculated in hours) spent by both group members working on the different chapters of the document.

- **Marcello Braghioli**

Chapter	Effort spent in hours
1	1
2	20
3	0
4	1
5	6

Table 6.1: Effort spent - Marcello Braghioli

- **Filippo Valentini**

Chapter	Effort spent in hours
1	1
2	20
3	0
4	2
5	3

Table 6.2: Effort spent - Filippo Valentini

7 | References

- RASD and DD assignment specification for the Software Engineering 2 course, a.y. 2024-2025.
- Slides of the Software Engineering 2 course, available on WeBeep.
- MondoDB Documentation: <https://www.mongodb.com/docs/>
- Pezzè, M. and Young, M. Software testing and analysis: process, principles, and techniques. John Wiley & Sons, 2008.
- Apache JMeter: <https://jmeter.apache.org>

List of Figures

2.1	High-level architectural schema	4
2.2	Component Diagram	8
2.3	Deployment Diagram	9
2.4	Student registration Sequence Diagram	11
2.5	Company registration Sequence Diagram	12
2.6	User login Sequence Diagram	13
2.7	CV upload Sequence Diagram	14
2.8	CV removal Sequence Diagram	15
2.9	Internship publication Sequence Diagram	16
2.10	Internship removal Sequence Diagram	17
2.11	CV selection Sequence Diagram	18
2.12	Internship selection Sequence Diagram	19
2.13	Match creation by a student Sequence Diagram	20
2.14	Match creation by a company Sequence Diagram	21
2.15	Interview set up Sequence Diagram	22
2.16	Selection process questionnaire compilation Sequence Diagram	23
2.17	Student admission Sequence Diagram	24
2.18	Student rejection Sequence Diagram	25
2.19	Student satisfaction questionnaire compilation Sequence Diagram	26
2.20	Company satisfaction questionnaire compilation Sequence Diagram	27
2.21	Student message providing Sequence Diagram	28
2.22	Company message providing Sequence Diagram	29
3.1	Login page UI	53
3.2	Student registration UI	54
3.3	Company registration UI	54
3.4	Student homepage UI	55
3.5	Student recommended internships UI	55
3.6	Student interested companies UI	56
3.7	Student interview calendar UI	57

3.8	Student messaging section UI	57
3.9	Student satisfaction questionnaires section UI	58
3.10	Company homepage UI	59
3.11	Company internship section UI	59
3.12	Company completed matches UI	60
5.1	First thread integration	72
5.2	Second thread integration	73
5.3	Third thread integration	73
5.4	Fourth thread integration	74
5.5	Fifth thread integration	75
5.6	Sixth thread integration	75
5.7	Seventh thread integration	76

List of Tables

2.1	Authentication Manager REST request - start student verification	30
2.2	Authentication Manager REST request - student verification	30
2.3	Authentication Manager REST request - student registration	30
2.4	Authentication Manager REST request - company registration	31
2.5	Authentication Manager REST request - user login	31
2.6	Student Manager REST request - CV upload	31
2.7	Student Manager REST request - CV removal	31
2.8	Student Manager REST request - recommended internships retrieval	32
2.9	Student Manager REST request - internship selection	32
2.10	Student Manager REST request - interested companies retrieval	32
2.11	Student Manager REST request - match creation	32
2.12	Student Manager REST request - calendar retrieval	32
2.13	Student Manager REST request - selection questionnaire compilation . . .	33
2.14	Student Manager REST request - satisfaction questionnaires retrieval . . .	33
2.15	Student Manager REST request - satisfaction questionnaire compilation . .	33
2.16	Student Manager REST request - messages retrieval	33
2.17	Student Manager REST request - message communication	33
2.18	Company Manager REST request - internship publication	34
2.19	Company Manager REST request - internship section retrieval	34
2.20	Company Manager REST request - internship removal	34
2.21	Company Manager REST request - CV selection	34
2.22	Company Manager REST request - match creation	35
2.23	Company Manager REST request - interview set up	35
2.24	Company Manager REST request - student admission	35
2.25	Company Manager REST request - student rejection	35
2.26	Company Manager REST request - satisfaction questionnaire compilation .	36
2.27	Company Manager REST request - messages retrieval	36
2.28	Company Manager REST request - message communication	36
2.29	Student Verification Platform REST request - redirect to platform	37

	List of Tables	83
2.30	Student Verification Platform REST request - student verification	37
2.31	Company Verification Platform REST request - company verification . . .	37
6.1	Effort spent - Marcello Braghiroli	78
6.2	Effort spent - Filippo Valentini	78